
AutoSGD: Automatic Learning Rate Selection for Stochastic Gradient Descent

Nikola Surjanovic
 Department of Statistics
 University of British Columbia
 Vancouver, BC V6T 1Z4
 nikola.surjanovic@stat.ubc.ca

Alexandre Bouchard-Côté
 Department of Statistics
 University of British Columbia
 Vancouver, BC V6T 1Z4
 bouchard@stat.ubc.ca

Trevor Campbell
 Department of Statistics
 University of British Columbia
 Vancouver, BC V6T 1Z4
 trevor@stat.ubc.ca

Abstract

The learning rate is an important tuning parameter for stochastic gradient descent (SGD) and can greatly influence its performance. However, appropriate selection of a learning rate schedule across all iterations typically requires a non-trivial amount of user tuning effort. To address this, we introduce AutoSGD: an SGD method that automatically determines whether to increase or decrease the learning rate at a given iteration and then takes appropriate action. We introduce theory supporting the convergence of AutoSGD, along with its deterministic counterpart for standard gradient descent. Empirical results suggest strong performance of the method on a variety of traditional optimization problems and machine learning tasks.

1 Introduction

Stochastic gradient descent (SGD) [RM51] and its many popular variants—such as AdaGrad [DHS11], RMSProp [HSS12], and Adam [KB14]—are indispensable tools for solving optimization problems using cheap, noisy gradient evaluations. For instance, such optimization problems are frequently encountered in machine learning and statistics when the amount of training data is large and each stochastic gradient evaluation only involves a small subset of the training data.

An important goal of modern optimization algorithms is to provide efficient convergence to an optimum of the objective function with as little tuning effort required by the user as possible. Gradient descent algorithms each differ in their set of tuning parameters, but the learning rate (sequence) is common among almost all such methods and is critical to performance. If the learning rate is chosen to be too large, SGD may become unstable or diverge; whereas if the learning rate is too small, the iterates may converge but at a painstakingly slow pace. Further, there is usually no “one size fits all” learning rate for a given optimization problem as different regions of the parameter space during optimization may benefit from larger or smaller learning rates.

In this work we introduce AutoSGD, a new SGD algorithm that adaptively selects appropriate learning rates on the fly by comparing the performance of neighbouring (larger and smaller) learning rates. The iterates are divided into episodes, during which we assess which of three learning rates is the winner. We also allow to reset to a previous point and consider a collection of smaller learning rates if we realize all learning rates are too large. We establish that both AutoSGD and its deterministic counterpart (i.e., based on exact gradients) converge under appropriate assumptions on the objective

function. In our experiments we find that AutoSGD outperforms or is comparable to other SGD methods with minimal or no tuning effort.

Related work. Several authors have developed diagnostics for detecting when SGD switches from the transient phase to the stationary phase. For instance, if stationarity is detected, the learning rate can be decreased (e.g., halved). The work of [Pf83, Pf90] considers inner products of consecutive gradients to detect when stationarity is reached; this work and follow-up methods by [CT17, CL20] apply to SGD both with and without momentum. Similar approaches are considered by [PDF20, Yai18, LXZ19], using different diagnostics to detect stationarity, and a related approach specific to variational inference is presented by [WAVH24]. However, a drawback of such methods is that they do not allow for an increase in the learning rate as they do not prescribe what action should be taken during the transient phase of SGD.

Another line of work considers line-search variants in the stochastic setting [GRS23, VML⁺19]. These methods require the specification of a maximum learning rate $\gamma_{\max}$ from which they backtrack until a stochastic version of standard Armijo conditions are satisfied. The methods and theoretical results are primarily designed for the interpolation setting. Also, it is not clear how the stochastic Armijo conditions perform when combined with control variates for variance reduction in gradient estimates [GSBR20] (e.g., such as the “sticking the landing” estimator, commonly used for black-box variational inference [KMG24, RWD17]). Furthermore, such methods are restricted by their choice of $\gamma_{\max}$, which can hinder performance if it is chosen to be too small.

Recently, hyperparameter-free methods similar in spirit to our proposed AutoSGD algorithm have emerged in both the deterministic and stochastic optimization settings [DM23, IHC23, KMJ23, OT17, KIH24, MD23, MM19, KJ24, LVLLJ21, OLJL22, AK24]. In particular, the distance over gradients (DoG) algorithm [IHC23], and a corresponding weighted gradient version (DoWG) [KMJ23], have proven to be empirically successful with theoretical convergence guarantees. Inspired by a result of [CH22], they consider an estimate of the distance to the optimum divided by a square root of a (weighted) sum of squared gradient norms. The DoWG method [KMJ23] modifies the original DoG [IHC23] algorithm and establishes convergence guarantees primarily in the deterministic setting. For optimizers such as D-Adaptation [DM23] (and DoG [IHC23]), [MD23] remark that “the methods might still be slow because the denominator in the step size might grow too large over time.”

2 Setup

The goal in this work is to find a minimum of a differentiable objective function $f : \mathbb{R}^d \rightarrow \mathbb{R}$. Without loss of generality, throughout we assume that f is nonnegative and attains its minimum at $\min_x f(x) = 0$, as none of our proposed methods require knowledge of the minimum value attained by f . We also assume that f and its gradient ∇f take the form

$$f(x) = \mathbb{E}[f(x, u)], \quad \nabla f(x) = \mathbb{E}[g(x, u)], \quad u \sim \phi,$$

for a noise distribution ϕ . For the purposes of our algorithms, we assume the ability to evaluate $g(x, u)$ exactly, and $f(x, u)$ up to an unknown constant independent of x but which may depend on u , i.e., we are able to evaluate $\tilde{f}$ where $f(x, u) = \tilde{f}(x, u) + C_u$. For simplicity in the rest of the paper, we will simply refer to $f(x, u)$. This setup and model of computation is typical of stochastic optimization problems where $f(x)$ and $\nabla f(x)$ are expensive or intractable to compute, e.g., the common empirical risk minimization setting where $f(x) = n^{-1} \sum_{i=1}^n f_i(x)$ if we set $u \sim \text{Unif}(\{1, 2, \dots, n\})$ and $f(x, u) = f_u$. However, note that it also generalizes the case of classical deterministic optimization problems where f and ∇f are tractable, as we can simply set $f(x, u) = f(x)$ and $g(x, u) = \nabla f(x)$ (i.e., ignore the noise variable).

Starting with an initial iterate $y_0 \in \mathbb{R}^d$, a sequence of learning rates $(\gamma_k)_{k \geq 0}$, and a noise sequence $(u_k)_{k \geq 0}$, the SGD algorithm is defined by the sequence $(y_k)_{k \geq 0}$ produced by the iteration

$$y_{k+1} = y_k - \gamma_k g(x_k, u_k).$$

3 Automatic learning rate selection

In this section we introduce AutoSGD, a stochastic gradient descent algorithm with automatic learning rate selection. To build intuition, we first present AutoGD, which is a deterministic version of AutoSGD useful for problems where evaluation of the full gradient is computationally feasible.

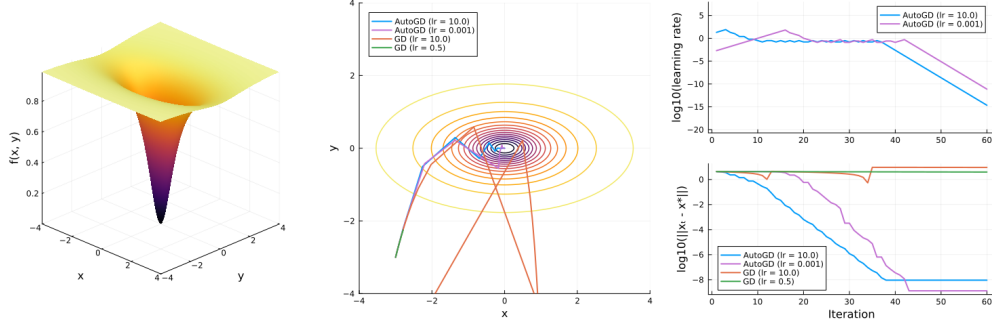


Figure 1: Performance of deterministic optimizers on the non-convex objective function $f(x, y) = 1 - 1/(1 + x^2 + 4y^2)$. **Left:** Surface plot of the objective function. **Middle:** Trajectories of AutoGD with initial learning rates $\gamma_0 \in \{0.001, 10.0\}$ and GD with learning rates $\gamma \in \{0.5, 10.0\}$ over 100 iterations. Here, GD with $\gamma = 0.5$ converges very slowly, while $\gamma = 10.0$ is unstable. AutoGD is stable as it approaches the minimum for different learning rate values. **Top right:** Automatically selected learning rates (on log scale) for each of the first 60 iterations. AutoGD automatically learns to anneal the learning rate in the initial phase, and then decreases the learning rate upon convergence. **Bottom right:** Distance to optimum (log scale) for AutoGD and GD iterates.

3.1 AutoGD

The AutoGD algorithm has the following inputs: an initial iterate x_0 , initial learning rate γ_0 , and learning rate expansion/contraction factors $0 < c < 1 < C < \infty$. Theorem 4.5 shows that AutoGD works with a wide range of reasonable values of C and c , but we recommend a default choice of $C = 1/c = 2$, which corresponds to doubling/halving the learning rate. We also note that even if γ_0 is initially too small or too large, we should reach an appropriate learning rate γ within $|\log_2(\gamma/\gamma_0)|$ iterations, and so γ_0 can be set anywhere within a wide acceptable neighbourhood.

Suppose that at time $t \geq 0$ our current iterate and learning rate are x_t and γ_t , respectively. We define the *proposed* new learning rate and iterate,

$$\gamma'_{t+1} = \arg \min_{\gamma \in \{c\gamma_t, \gamma_t, C\gamma_t\}} f(x_t - \gamma \nabla f(x_t)), \quad x'_{t+1} = x_t - \gamma'_{t+1} \nabla f(x_t).$$

(In case the $\arg \min$ over γ is not unique, we choose the largest γ .) Then we either accept or reject the proposal as follows:

$$(x_{t+1}, \gamma_{t+1}) = \begin{cases} (x'_{t+1}, \gamma'_{t+1}), & f(x'_{t+1}) < f(x_t) \\ (x_t, c \cdot \gamma_t), & f(x'_{t+1}) \geq f(x_t) \end{cases}.$$

That is, we look forward one step to see which learning rate in $\{c\gamma_t, \gamma_t, C\gamma_t\}$ would perform best, and then use the best choice to obtain x_{t+1} . Because there is no noise in the deterministic setting, if all learning rates in the grid fail to decrease the objective, we decrease the learning rate by a factor of c to consider a new grid of learning rates. (In the common case where $C = 1/c$, we can decrease by c^2 to try a fresh collection of smaller learning rates.)

Fig. 1 shows the performance of AutoGD on a non-convex objective function and how the chosen learning rate varies as the algorithm proceeds. In terms of the computational complexity of AutoGD compared to standard gradient descent, the main difference between the two algorithms is that the former evaluates three learning rates at each iteration instead of one. However, these evaluations can be performed in parallel and independently from one another. Therefore, with an efficient implementation, each iteration can have the same average runtime as GD, but requires a constant factor of three times as much memory. Appendix E.4 contains additional experimental results that show excellent performance of AutoGD on various tasks, along with a comparison to backtracking line search.

3.2 AutoSGD

AutoSGD generalizes AutoGD to the setting where we have access only to noisy estimates of the gradient and objective function. Given a learning rate grid $\{c\gamma, \gamma, C\gamma\}$, the goal is to determine

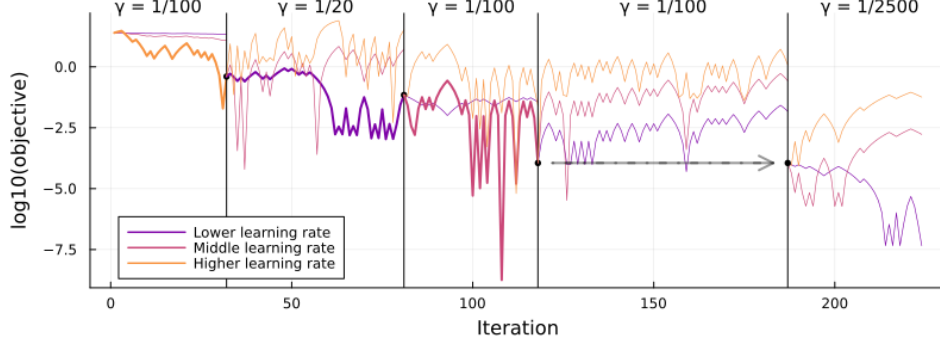


Figure 2: Example of the AutoSGD learning rate selection procedure with $C = 1/c = 5$ (a larger value of C is used for better visualization). Episode endpoints are indicated as black vertical lines and the middle learning rate within the episode is indicated at the top of each section. The exact objective function value is known at episode endpoints here, but is typically estimated in practice. Bold lines indicate winning trajectories. **Episode 0:** The highest learning rate is selected for the next episode. **Episode 1:** The smallest learning rate is selected. **Episode 2:** The middle learning rate is selected. **Episode 3:** Evidence of function increase at all learning rates. Decrease by $1/C^2$ and restart at the previous episode’s starting point x_{t-1} . **Episode 4:** In progress.

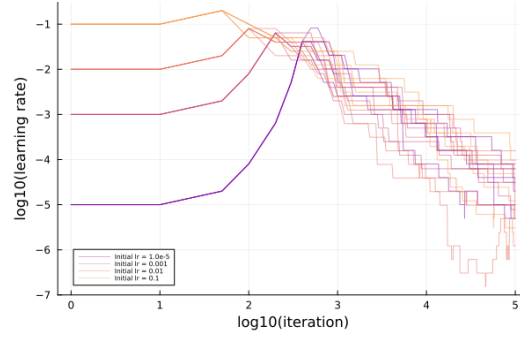


Figure 3: Performance of the new learning rate selection procedure within AutoSGD (plotted on log-log scale) on the “sum of quadratics” problem. AutoSGD is initialized with four different learning rates $\gamma \in \{10^{-1}, 10^{-2}, 10^{-3}, 10^{-5}\}$. All three initializations learn to automatically warm up the learning rate and eventually converge and decay at a rate of approximately $O(1/t)$ in this example.

which learning rate yields the lowest objective value. Stochasticity in f and ∇f must be handled, and so we introduce a decision process. This results in random lengths of time, called *episodes*, where the learning rate grid is constant until a decision is made, at which point the grid may shift.

To assess the performance of the three different learning rates, we keep track of three streams of SGD iterates (Fig. 2). Consider an initial point x_t and learning rate γ_t at the start of episode t . We run three standard streams $k = 1, 2, \dots$ of SGD during the episode:

$$\begin{aligned}\underline{x}_{t,k+1} &= \underline{x}_{t,k} - c\gamma_t g(\underline{x}_{t,k}, u_{t,k}) \\ x_{t,k+1} &= x_{t,k} - \gamma_t g(x_{t,k}, u_{t,k}) \\ \bar{x}_{t,k+1} &= \bar{x}_{t,k} - C\gamma_t g(\bar{x}_{t,k}, u_{t,k}),\end{aligned}$$

which are each initialized according to $\underline{x}_{t,0} = x_{t,0} = \bar{x}_{t,0} = x_t$. These three streams all stop at a (random) time τ_t when we have confidently identified what to do for the next episode (e.g., shift the grid to smaller learning rates if the stream $\underline{x}_{t,k}$ clearly outperforms the others).

We now describe the four valid moves at the end of an episode. Let I_t , S_t , D_t , and R_t be binary indicators denoting the decisions to increase γ , stay at γ , decrease γ , or restart to the previous episode’s starting point while decreasing the learning rate. More precisely, at the end of episode t with initial learning rate γ_t , we may either: increase the learning rate (event $I_t = 1$), with $(x_{t+1}, \gamma_{t+1}) = (\bar{x}_{t,\tau_t}, C\gamma_t)$; decrease the learning rate ($D_t = 1$), with $(x_{t+1}, \gamma_{t+1}) = (\underline{x}_{t,\tau_t}, c\gamma_t)$;

Algorithm 1 AutoSGD

Require: Initial state x_0 , initial learning rate γ_0 , number of episodes T , expansion coefficient $C > 1$ (default: $C = 2$), contraction coefficient $0 < c < 1$ (default: $c = 1/2$).

- 1: **for** t in $0, 1, \dots, T - 1$ **do**
- 2: $\tau_t \leftarrow 0, \quad \underline{x}_{t,0}, x_{t,0}, \bar{x}_{t,0} \leftarrow x_t, \quad I_t, D_t, S_t, R_t \leftarrow 0$
- 3: **while** $I_t = D_t = S_t = R_t = 0$ **do** ▷ run three SGD streams in parallel
- 4: $\underline{x}_{t,\tau_t+1} \leftarrow \underline{x}_{t,\tau_t} - c\gamma_t g(\underline{x}_{t,\tau_t}, u_{t,\tau_t})$
- 5: $x_{t,\tau_t+1} \leftarrow x_{t,\tau_t} - \gamma_t g(x_{t,\tau_t}, u_{t,\tau_t})$
- 6: $\bar{x}_{t,\tau_t+1} \leftarrow \bar{x}_{t,\tau_t} - C\gamma_t g(\bar{x}_{t,\tau_t}, u_{t,\tau_t})$
 ▷ Algorithm 2, see supplement for constant-memory/time decision() implementation
- 7: $I_t, D_t, S_t, R_t \leftarrow \text{decision}((\underline{x}_{t,k})_{k=0}^{\tau_t}, (x_{t,k})_{k=0}^{\tau_t}, (\bar{x}_{t,k})_{k=0}^{\tau_t})$
- 8: $\tau_t \leftarrow \tau_t + 1$
- 9: **end while**
- 10: **if** $I_t = 1$, set $x_{t+1}, \gamma_{t+1} \leftarrow \bar{x}_{t,\tau_t}, C\gamma_t$ ▷ increase learning rate
- 11: **elseif** $D_t = 1$, set $x_{t+1}, \gamma_{t+1} \leftarrow \underline{x}_{t,\tau_t}, c\gamma_t$ ▷ decrease learning rate
- 12: **elseif** $S_t = 1$, set $x_{t+1}, \gamma_{t+1} \leftarrow x_{t,\tau_t}, \gamma_t$ ▷ stay at the same learning rate
- 13: **else** set $x_{t+1}, \gamma_{t+1} \leftarrow x_t, c\gamma_t$ ▷ restart: objective increase detected at all learning rates
- 14: **endif**
- 15: **end for**
- 16: **return** x_T

stay at the same learning rate ($S_t = 1$), with $(x_{t+1}, \gamma_{t+1}) = (x_{t,\tau_t}, \gamma_t)$; or detect that all learning rates result in an increase in the objective ($R_t = 1$), in which case $(x_{t+1}, \gamma_{t+1}) = (x_t, c\gamma_t)$. Fig. 2 displays all four possible scenarios on a simple example.

The indicators (I_t, D_t, S_t, R_t) are defined through a decision process. We leave them in a general form for our initial presentation and define one possible approach that can be computed efficiently with constant memory in the following section. The AutoSGD algorithm is presented in Algorithm 1.

3.2.1 A decision process

The t^{th} episode length τ_t is random and given the three SGD streams $(\underline{x}_{t,j})_{j=0}^k, (x_{t,j})_{j=0}^k, (\bar{x}_{t,j})_{j=0}^k$ up to time k , we must decide whether to keep running the streams. If not, we must decide whether to increase the learning rate, decrease the learning rate, stay at the same learning rate, or reject all learning rates in the grid and restart at the previous episode's iterate.

We specify one possible stopping time that can be easily computed online with constant memory cost. We introduce two new groups of mutually independent sources of noise for learning rate assessment: $u_{t,k}^{(1)}, u_{t,k}^{(2)}$, which are also independent of the gradient sources of noise $u_{t,k}$. Within episode t at sub-episode iteration k , for $m = 1, 2$ define the differences

$$\begin{aligned}\underline{\Delta}_{t,k}^{(m)} &= f(x_t, u_{t,k}^{(m)}) - f(\underline{x}_{t,k}, u_{t,k}^{(m)}) \\ \Delta_{t,k}^{(m)} &= f(x_t, u_{t,k}^{(m)}) - f(x_{t,k}, u_{t,k}^{(m)}) \\ \bar{\Delta}_{t,k}^{(m)} &= f(x_t, u_{t,k}^{(m)}) - f(\bar{x}_{t,k}, u_{t,k}^{(m)}).\end{aligned}$$

Intuitively, a learning rate should perform well if the long-run averages of $\Delta_{t,k}^{(m)}$ are large relative to their estimated variance. To perform this assessment, we define

$$Z_{t,k} = \frac{\sum_{j=0}^k \frac{1}{2} (\Delta_{t,j}^{(1)} + \Delta_{t,j}^{(2)})}{\sqrt{\sum_{j=0}^k \max\left(\epsilon, \frac{1}{2} (\Delta_{t,j}^{(1)} - \Delta_{t,j}^{(2)})^2\right)}},$$

with $\epsilon = 10^{-12}$, to avoid division by zero in the rare cases where $\Delta_{t,k}^{(1)} \approx \Delta_{t,k}^{(2)}$ up to floating point error. The statistics $\underline{Z}_{t,k}$ and $\bar{Z}_{t,k}$ for the lower and upper streams are defined analogously using $\underline{\Delta}_{t,k}^{(m)}$ and $\bar{\Delta}_{t,k}^{(m)}$. Crucially, we note that the $Z_{t,k}$ can be computed online with constant memory cost.

We use these statistics inside Algorithm 2, detailed in the supplement, where we also describe the constant-memory online implementation along with the exact conditions that a decision process should satisfy in order to guarantee AutoSGD convergence.

4 Theory

In this section we prove various properties of AutoGD and AutoSGD. Standard definitions, such as (strong) convexity and L -smoothness are deferred to the supplementary material.

4.1 AutoGD

The function evaluations of iterates produced by AutoGD must always converge. This means that AutoGD will never create unstable or divergent iterations, even under very weak assumptions on f .

Proposition 4.1. *Let f be differentiable. The iterates $f(x_t)$ of AutoGD converge (not necessarily to a local minimum): $\lim_{t \rightarrow \infty} f(x_t) = \underline{f}$, for some $\underline{f} \geq 0$.*

The following result highlights why it is important to consider a “no movement” step in the algorithm, where we decrease the learning rate and restart from the previous value of x_t . Suppose we were to set

$$x_{t+1} = x_t - \gamma \nabla f(x_t), \quad \text{for some } \gamma \in \{c\gamma_t, \gamma_t, C\gamma_t\} \quad (1)$$

at every iteration, even if objective function increase is detected at all γ values. Then the algorithm would diverge even on some simple polynomials. The following results assumes that in our AutoGD algorithm we shrink the middle learning rate by c if we detect an increase in the objective at all learning rates in the grid. Recall that in practice, if $C = c^{-1}$, we can shrink by c^2 . A similar result to the one below, with slightly modified constants, holds also in that special case.

Proposition 4.2. *Consider iterates $(x_t)_{t \geq 0}$ of AutoGD with initial learning rate γ_0 and where at each iteration we are forced to update our iterates according to (1) (i.e., without the “no movement” option). Suppose $|x_0| > 1$ and that $f(x) = x^{2p}$ for $p \geq 2$ with*

$$p > \frac{c+1}{2c^2\gamma_0|x_0|}.$$

Then the iterates diverge exponentially: $|x_t| = \Omega(c^{-t})$.

In contrast, we know that our proposed version of AutoGD (with a “no movement” option) always converges by Proposition 4.1. Next, we establish conditions under which AutoGD converges to a minimum with a bound on the rate of convergence. For $x \in \mathbb{R}^d$ with $\|\nabla f(x)\| \neq 0$ and $\gamma > 0$, define

$$G(x, \gamma) = \frac{\nabla f(x)}{\|\nabla f(x)\|}^T \int_0^1 2(1-t) \nabla^2 f(x - t\gamma \nabla f(x)) dt \frac{\nabla f(x)}{\|\nabla f(x)\|}, \quad (2)$$

which is used to control the rate of descent. To state the result, we introduce a few assumptions.

Assumption 4.3. *For all $x \in \mathbb{R}^d$ with $\|\nabla f(x)\| \neq 0$, $f(x - \gamma \nabla f(x))$ is unimodal (with a unique minimum) for $\gamma > 0$.*

Assumption 4.4. *f is L -smooth, twice continuously differentiable, and there exists $\bar{\gamma} > 1/L$ such that for all x with $\|\nabla f(x)\| \neq 0$, we have $\bar{\gamma} - \frac{\bar{\gamma}^2}{2} G(x, \bar{\gamma}) \leq \frac{1}{2L}$.*

We remark that Assumption 4.3 is satisfied in the case where f is μ -strongly convex by definition. In this case, then the technical condition in Assumption 4.4 also follows by noting that $G \geq \mu$, by strong-convexity, and setting $\bar{\gamma} \geq (1 + \sqrt{1 - 1/\kappa})/\mu$, where $\kappa = L/\mu$.

Theorem 4.5. *Suppose that f is μ -Polyak-Łojasiewicz and Assumptions 4.3 and 4.4 hold. Let*

$$\tau = \left\lceil \log_{c^{-1}} \frac{Lc^{-1}\bar{\gamma}}{\sqrt{2}-1} \right\rceil + \max \left\{ \left\lceil \log_C \frac{C^{-1}}{c} \right\rceil, \left\lceil \log_{c^{-1}} \frac{C}{c^{-1}} \right\rceil \right\} + 1$$

$$t_0 = \max \left\{ 0, \log_{c^{-1}} \frac{\gamma_0}{\bar{\gamma}}, \log_C \frac{(\sqrt{2}-1)}{L\gamma_0} \right\}.$$

Then

$$f(x_t) \leq f(x_0) \cdot \left(1 - \frac{\mu c(\sqrt{2}-1)(2C - c(\sqrt{2}-1))}{C^2 L} \right)^{\lfloor \frac{t-t_0}{\tau} \rfloor}.$$

From this result, we can see that regardless of the choice of initial learning rate, we can obtain a linear convergence rate of approximately $1/(\kappa \log \kappa)$. In contrast, optimally-tuned gradient descent has rate $1/\kappa$, but requires selecting the learning rate depending on the unknown quantity L .

4.2 AutoSGD

We present a result that establishes AutoSGD convergence under some assumptions. The exact conditions are described precisely in Appendix C.2 and a statement of the following theorem with exact constants is also provided there. The conditions roughly stipulate that: the indicators I_t, S_t, D_t are more likely to trigger when their respective objective function is lower; if the learning rate is too large, there is a high probability of restarting ($R_t = 1$); if the learning rate is too small, there is a high probability of increasing the learning rate quickly ($I_t = 1$); and if the learning rate is adequate, the test takes some time to trigger in order to guarantee a certain amount of descent. Theorem 4.6 demonstrates that AutoSGD converges under these assumptions, and that the rate is linear at a sequence of random episode iterations. It is worth noting that the proof in Appendix C.2 uses techniques drawn from the Markov chain Monte Carlo literature that do not commonly appear in stochastic optimization, which may be useful for future study of similar adaptive algorithms.

Theorem 4.6. *Fix a function $f : \mathbb{R}^d \rightarrow \mathbb{R}$ and suppose that the conditions listed in Appendix C.2 hold. Then there exists a random sequence of times t_n , with $t_n < \infty$ almost surely, and constants $0 \leq \nu < 1$ and $0 \leq f_0, a, b < \infty$, such that*

$$\forall n \in \mathbb{N}_0, \quad \mathbb{E}[f(x_{t_n})] \leq \nu^n f_0 \quad \text{and} \quad \mathbb{E}[t_n] \leq a + bn.$$

Recall that each x_t is the start of a new *episode*, whose length (in terms of individual SGD iterations) is τ_t . While the above result demonstrates linear convergence in episode iterations, it does not make any claim about the total number of SGD iterations. Indeed, the technical assumptions in Appendix C.2 indicate that τ_t should increase as γ_t decreases, which results in an overall sublinear convergence as expected for SGD with a decaying learning rate in the presence of noise.

5 Experiments

We assess the performance of AutoSGD on both classical optimization objectives and various machine learning (ML) training experiments. We consider five classes of optimizers: SGD with various learning rates held constant, SGD with various initial learning rates and decay, Distance over Gradient (DoG) [IHC23], schedule-free SGD [DYK⁺24], non-monotone line-search (NMLS) [GRS23], and AutoSGD. For NMLS, we use backtracking with the modified termination condition in [GRS23], but do not modify the initial learning rate as it requires knowing quantities such as $\inf f_i$. The classical optimization experiments are performed in Julia [BEKS17], while the remaining ML experiments are done in Python within the PyTorch [PGM⁺19] framework. All experiments are performed on the ARC Sockeye compute cluster at the University of British Columbia. Experiments with AutoGD performed on deterministic optimization problems [SB13] are presented in the supplement. In all plots, we define one *iteration* (x-axis) such that all methods incur roughly the same computational cost per iteration (i.e., we do not plot x_t but rather all iterations within all episodes).

5.1 Classical optimization

We consider five classical optimization settings: linear regression, approximate matrix factorization, logistic regression, sums of quadratic functions, and multiclass logistic regression. The last four settings are based on [VML⁺19] and [NLS22]. Detailed descriptions of the experimental settings and additional figures are provided in the supplement. In this section we present figures for the first three groups of experiments.

Fig. 3 shows a representative example of the learning rates selected by AutoSGD. We find that AutoSGD is able to successfully adapt the learning rate on various problems, automatically introducing an initial warmup and decay of the learning rate (more examples shown in the supplement).

In Fig. 4 we present results for choices of initial learning rates that result in best loss decay for the majority of methods (excluding AutoSGD and DoG). To highlight the robustness of AutoSGD to a specification of initial learning rate, in Fig. 5 we present results for all optimizers with a smaller

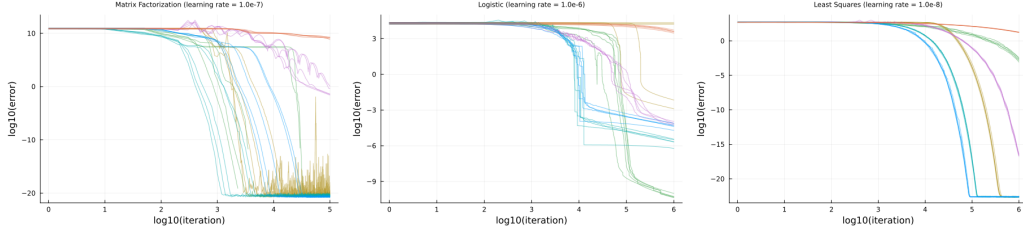


Figure 4: Classical optimization settings with learning rates chosen that are favourable to the majority of optimizers (excluding AutoSGD and DoG). Five different seeds are presented for each optimizer. The optimizers are labeled with the following colours: AutoSGD, DoG, SFSGD, SGD (constant), SGD (invsqrt), NMLS. The matrix factorization example corresponds to $k = 10$.

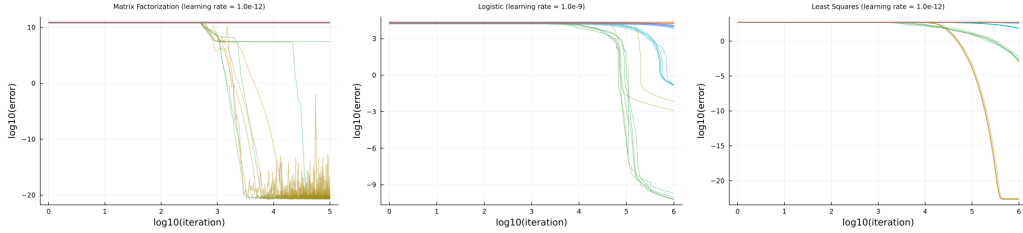


Figure 5: Classical optimization settings with initially small learning rates for the different optimizers in order to assess robustness.

initial learning rate outside of its approximately optimal magnitude. Additional experimental results are presented in Figs. 10 to 12. In all settings, AutoSGD and DoG perform well without the need for any tuning. Generally, we find that AutoSGD has more stable performance once it reaches an optimum (even without any averaging), whereas DoG can exhibit some noise possibly due to the gradient noise. This is seen, for instance, in the matrix factorization example, where the DoG iterates display a considerable amount of noise towards the end of optimization.

In Fig. 4 with tuned learning rates, we find that NMLS can perform quite well on certain problems. However, because its maximum learning rate is capped, if an initial guess is too low, its performance is severely hindered, as shown in Fig. 5. We note that in all experiments we set the maximum learning rate for NMLS to 10 times the amount specified for other optimizers in order for it to be able to explore a wider range of learning rates. However, even with this additional room, its performance still suffers if the initial guess is too small. We note that on the figures the iteration count is not scaled with respect to the number of NMLS backtracks, and so we would expect NMLS to perform *a constant factor worse* than displayed here.

In the multiclass logistic regression setting (Figs. 10 and 11), we found that AutoSGD can sometimes momentarily be unstable before reverting back to a desirable loss. Somewhat surprisingly, even with this temporary instability, the final training loss is still better than all other methods for most of the seeds.

Finally, we find that on some of these problems, such as the matrix factorization and sum of quadratics problems, schedule-free SGD can exhibit oscillatory behaviour. Interestingly, for the ML training tasks in the following section, this oscillatory behaviour does not occur, presumably due to higher levels of gradient noise.

5.2 ML training

For our ML experiments, we consider both natural language processing (NLP) and vision tasks. We fine-tune base models on various datasets, allowing us to remain within an academic compute budget. For the NLP tasks, we fine-tune a RoBERTa-base model [LOG⁺19] with different datasets from from the GLUE benchmark [WSM⁺19]: QNLI (question answering) [RZLL16], MRPC (semantic equivalence) [DB05], and SST-2 (sentiment analysis) [SPW⁺13]. For the vision tasks, we fine-tune a ResNet [HZRS16] on the CIFAR-10 and CIFAR-100 data sets [Kri09]. This experimental setup is similar to some of those considered in [IHC23]. Based on the results from Section 5.1, we exclude

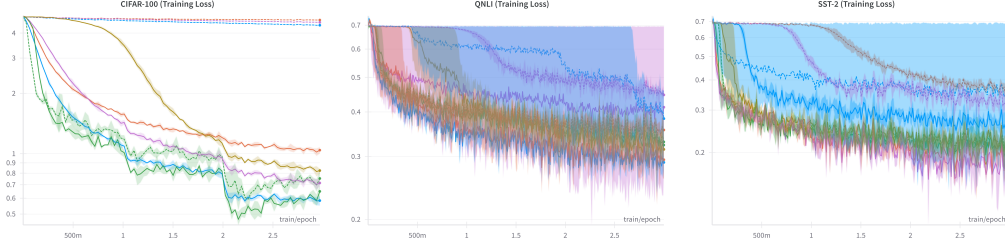


Figure 6: Training loss for various optimizers on three out of five ML data sets. Solid lines represent averages across seeds, and shaded regions are the maximum and minimum across three seeds for a given optimizer. For each data set and optimizer configuration, we present the best (solid line) and worst (dashed line) learning rate selections from a predefined grid. The colour scheme is the same as in Fig. 4: AutoSGD, DoG, SFGSD, SGD (constant), SGD (invsqrt).

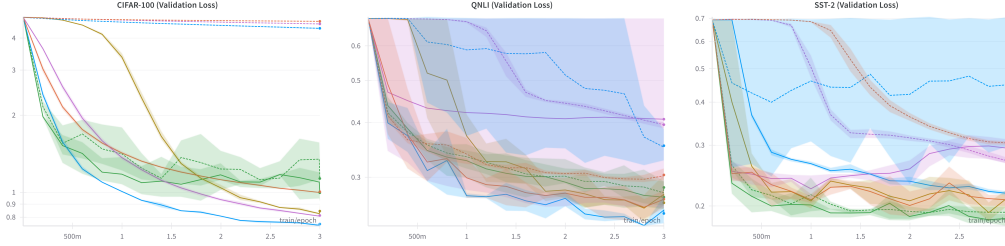


Figure 7: Validation loss for various optimizers on three out of five ML data sets.

optimizers that are not robust to the choice of initial learning rate, with the exception of SGD and schedule-free SGD as they are both widely used. For these experiments, we consider learning rates $\gamma \in \{10^{-1}, 10^{-2}, 10^{-3}\}$, which seem to be an acceptable range for this problem set, and then keep the best and worst performer for each optimizer that has a learning rate parameter.

Figs. 6 and 7 present the training and validation loss, respectively, for three out of the five settings (CIFAR-100, QNLI, SST-2), with the remaining two (CIFAR-10, MRPC) presented in Figs. 13 and 14. From these results we take away the following: AutoSGD and DoG are both robust and perform well across different settings with minimal or no tuning. Between these two, which one performs better on a given task is problem-specific, and so we view both as useful optimizers in the ML training toolkit. We find that AutoSGD can sometimes be slightly sensitive to the initial learning rate parameter choice, but generally performs quite well. This is in contrast to schedule-free SGD, which can be highly sensitive to the choice of learning rate parameter. However, when the learning rate is chosen carefully for schedule-free SGD, it can sometimes significantly outperform both AutoSGD and DoG, while a poorly chosen learning rate can lead to considerably weaker training and validation loss.

6 Discussion

In this paper we presented AutoSGD, an algorithm for stochastic gradient descent with automatic selection of learning rates. In our experiments, we found that AutoSGD is robust to the choice of initial learning rate and outperforms or is on par with its competitors on many optimization problems. These experimental results are also supported by convergence theory for both AutoSGD and its deterministic counterpart, AutoGD. We note that the convergence proof for AutoSGD used techniques drawn from the Markov chain Monte Carlo literature, which may be useful for future developments and analyses.

Naturally, some directions for future work remain. Extensions to a learning rate proposal grid with more than three elements would raise interesting questions about an optimal number of streams and the choice of spacing between learning rates. Further, in this work we present the AutoSGD algorithm with a general decision process and consider one possible implementation of such a process. Exploring other decision processes and studying their properties would be a useful component of future work.

Acknowledgments and Disclosure of Funding

ABC and TC acknowledge the support of an NSERC Discovery Grant. NS acknowledges the support of a Four Year Doctoral Fellowship from the University of British Columbia. We additionally acknowledge use of the ARC Sockeye computing platform from the University of British Columbia.

References

- [AK24] Amit Attia and Tomer Koren. How free is parameter-free stochastic optimization? *arXiv:2402.03126*, 2024.
- [Apo67] Tom Apostol. *Calculus*, volume 1. John Wiley & Sons, 2nd edition, 1967.
- [AS23] Kyriakos Axiotis and Maxim Sviridenko. Gradient descent converges linearly for logistic regression on separable data. In *International Conference on Machine Learning*, pages 1302–1319. PMLR, 2023.
- [BEKS17] Jeff Bezanson, Alan Edelman, Stefan Karpinski, and Viral B Shah. Julia: A fresh approach to numerical computing. *SIAM Review*, 59(1):65–98, 2017.
- [CH22] Yair Carmon and Oliver Hinder. Making SGD parameter-free. In *Conference on Learning Theory*, pages 2360–2389. PMLR, 2022.
- [CL20] Jerry Chee and Ping Li. Understanding and detecting convergence for stochastic gradient descent with momentum. In *2020 IEEE International Conference on Big Data*, pages 133–140. IEEE, 2020.
- [CT17] Jerry Chee and Panos Toulis. Convergence diagnostics for stochastic gradient descent with constant step size. *arXiv:1710.06382*, 2017.
- [DB05] Bill Dolan and Chris Brockett. Automatically constructing a corpus of sentential paraphrases. In *Third International Workshop on Paraphrasing*, 2005.
- [DCA⁺20] Akash Kumar Dhaka, Alejandro Catalina, Michael R Andersen, Måns Magnusson, Jonathan Huggins, and Aki Vehtari. Robust, accurate stochastic optimization for variational inference. *Advances in Neural Information Processing Systems*, 33:10961–10973, 2020.
- [DHS11] John Duchi, Elad Hazan, and Yoram Singer. Adaptive subgradient methods for online learning and stochastic optimization. *Journal of Machine Learning Research*, 12(7), 2011.
- [DM23] Aaron Defazio and Konstantin Mishchenko. Learning-rate-free learning by D-Adaptation. In *International Conference on Machine Learning*, pages 7449–7479. PMLR, 2023.
- [DMPS18] Randal Douc, Eric Moulines, Pierre Priouret, and Philippe Soulier. *Markov Chains*. Springer, 2018.
- [Dur10] Rick Durrett. *Probability: Theory and Examples*. Cambridge University Press, 4th edition, 2010.
- [DYK⁺24] Aaron Defazio, Xingyu Yang, Ahmed Khaled, Konstantin Mishchenko, Harsh Mehta, and Ashok Cutkosky. The road less scheduled. *Advances in Neural Information Processing Systems*, 37:9974–10007, 2024.
- [GCS⁺13] Andrew Gelman, John B Carlin, Hal S Stern, David B Dunson, Aki Vehtari, and Donald B Rubin. *Bayesian Data Analysis*. CRC Press, 2013.
- [GG23] Guillaume Garrigos and Robert M Gower. Handbook of convergence theorems for (stochastic) gradient methods. *arXiv:2301.11235*, 2023.
- [GR92] Andrew Gelman and Donald B Rubin. Inference from iterative simulation using multiple sequences. *Statistical science*, 7(4):457–472, 1992.
- [GRS23] Leonardo Galli, Holger Rahut, and Mark Schmidt. Don’t be so monotone: Relaxing stochastic line search in over-parameterized models. *Advances in Neural Information Processing Systems*, 36:34752–34764, 2023.
- [GSBR20] Robert M Gower, Mark Schmidt, Francis Bach, and Peter Richtárik. Variance-reduced methods for machine learning. *Proceedings of the IEEE*, 108(11):1968–1983, 2020.

- [HSS12] Geoffrey Hinton, N. Srivastava, and K. Swersky. Neural networks for machine learning lecture 6a: Overview of mini-batch gradient descent, 2012.
- [HZRS16] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 770–778, 2016.
- [IHC23] Maor Ivgi, Oliver Hinder, and Yair Carmon. DoG is SGD’s best friend: A parameter-free dynamic step size schedule. In *International Conference on Machine Learning*, pages 14465–14499. PMLR, 2023.
- [JKK⁺18] Prateek Jain, Sham M Kakade, Rahul Kidambi, Praneeth Netrapalli, and Aaron Sidford. Parallelizing stochastic gradient descent for least squares regression: mini-batching, averaging, and model misspecification. *Journal of Machine Learning Research*, 18(223):1–42, 2018.
- [KB14] Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv:1412.6980*, 2014.
- [KIHC24] Itai Kreisler, Maor Ivgi, Oliver Hinder, and Yair Carmon. Accelerated parameter-free stochastic optimization. In *The Thirty Seventh Annual Conference on Learning Theory*, pages 3257–3324. PMLR, 2024.
- [KJ24] Ahmed Khaled and Chi Jin. Tuning-free stochastic optimization. *arXiv:2402.07793*, 2024.
- [KMG24] Kyurae Kim, Yian Ma, and Jacob Gardner. Linear convergence of black-box variational inference: Should we stick the landing? In *International Conference on Artificial Intelligence and Statistics*, pages 235–243. PMLR, 2024.
- [KMJ23] Ahmed Khaled, Konstantin Mishchenko, and Chi Jin. DoWG unleashed: An efficient universal parameter-free gradient descent method. *Advances in Neural Information Processing Systems*, 36:6748–6769, 2023.
- [Kri09] Alex Krizhevsky. Learning multiple layers of features from tiny images. Technical report, University of Toronto, 2009.
- [LOG⁺19] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. RoBERTa: A robustly optimized BERT pretraining approach. *arXiv:1907.11692*, 2019.
- [LVLLJ21] Nicolas Loizou, Sharan Vaswani, Issam Hadj Laradji, and Simon Lacoste-Julien. Stochastic polyak step-size for SGD: An adaptive learning rate for fast convergence. In *International Conference on Artificial Intelligence and Statistics*, pages 1306–1314. PMLR, 2021.
- [LXZ19] Hunter Lang, Lin Xiao, and Pengchuan Zhang. Using statistics to automate stochastic optimization. *Advances in Neural Information Processing Systems*, 32, 2019.
- [MD23] Konstantin Mishchenko and Aaron Defazio. Prodigy: An expeditiously adaptive parameter-free learner. *arXiv:2306.06101*, 2023.
- [MM19] Yura Malitsky and Konstantin Mishchenko. Adaptive gradient descent without descent. *arXiv:1910.09529*, 2019.
- [MNR19] Nicole Mücke, Gergely Neu, and Lorenzo Rosasco. Beating SGD saturation with tail-averaging and minibatching. *Advances in Neural Information Processing Systems*, 32, 2019.
- [NLS22] Julie Nutini, Issam Laradji, and Mark Schmidt. Let’s make block coordinate descent converge faster: Faster greedy rules, message-passing, active-set complexity, and super-linear convergence. *Journal of Machine Learning Research*, 23(131):1–74, 2022.
- [OLJL22] Antonio Orvieto, Simon Lacoste-Julien, and Nicolas Loizou. Dynamics of SGD with stochastic Polyak stepsizes: Truly adaptive variants and convergence to exact solution. *Advances in Neural Information Processing Systems*, 35:26943–26954, 2022.
- [OT17] Francesco Orabona and Tatiana Tommasi. Training deep networks without learning rates through coin betting. *Advances in Neural Information Processing Systems*, 30, 2017.

- [PDF20] Scott Pesme, Aymeric Dieuleveut, and Nicolas Flammarion. On convergence-diagnostic based step sizes for stochastic gradient descent. In *International conference on machine learning*, pages 7641–7651. PMLR, 2020.
- [Pfl83] Georg Pflug. On the determination of the step size in stochastic quasigradient methods. Technical report, International Institute for Applied Systems Analysis, 1983.
- [Pfl90] Georg Pflug. Non-asymptotic confidence bounds for stochastic approximation algorithms with constant step size. *Monatshefte für Mathematik*, 110:297–314, 1990.
- [PGM⁺19] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, volume 32, 2019.
- [res25] ResNet-18. <https://huggingface.co/microsoft/resnet-18>, 2025.
- [RM51] Herbert Robbins and Sutton Monro. A stochastic approximation method. *The Annals of Mathematical Statistics*, pages 400–407, 1951.
- [rob25] RoBERTa base model. <https://huggingface.co/FacebookAI/roberta-base>, 2025.
- [Rou19] Nicolas Le Roux. Anytime tail averaging. *arXiv:1902.05083*, 2019.
- [RSS11] Alexander Rakhlin, Ohad Shamir, and Karthik Sridharan. Making gradient descent optimal for strongly convex stochastic optimization. *arXiv:1109.5647*, 2011.
- [RWD17] Geoffrey Roeder, Yuhuai Wu, and David K Duvenaud. Sticking the landing: Simple, lower-variance gradient estimators for variational inference. *Advances in Neural Information Processing Systems*, 30, 2017.
- [RZLL16] Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. SQuAD: 100,000+ questions for machine comprehension of text. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, pages 2383–2392, 2016.
- [SB13] Sonja Surjanovic and Derek Bingham. Virtual library of simulation experiments: Test functions and datasets, 2013.
- [SPW⁺13] Richard Socher, Alex Perelygin, Jean Wu, Jason Chuang, Christopher D Manning, Andrew Y Ng, and Christopher Potts. Recursive deep models for semantic compositionality over a sentiment treebank. In *Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing*, pages 1631–1642, 2013.
- [SZ13] Ohad Shamir and Tong Zhang. Stochastic gradient descent for non-smooth optimization: Convergence results and optimal averaging schemes. In *International Conference on Machine Learning*, pages 71–79. PMLR, 2013.
- [VML⁺19] Sharan Vaswani, Aaron Mishkin, Issam Laradji, Mark Schmidt, Gauthier Gidel, and Simon Lacoste-Julien. Painless stochastic gradient: Interpolation, line-search, and convergence rates. *Advances in Neural Information Processing Systems*, 32, 2019.
- [WAVH24] Manushi Welandawe, Michael Riis Andersen, Aki Vehtari, and Jonathan H Huggins. A framework for improving the reliability of black-box variational inference. *Journal of Machine Learning Research*, 25(219):1–71, 2024.
- [WSM⁺19] Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R Bowman. GLUE: A multi-task benchmark and analysis platform for natural language understanding. In *International Conference on Learning Representations*, 2019.
- [Yai18] Sho Yaida. Fluctuation-dissipation relations for stochastic gradient descent. *arXiv:1810.00004*, 2018.

A Common definitions

Some standard definitions are listed below for completeness.

Definition A.1. (*Convex function*) A function $f : \mathbb{R}^d \rightarrow \mathbb{R}$ is convex if for any $x, y \in \mathbb{R}^d$ and $t \in [0, 1]$ we have

$$f(tx + (1-t)y) \leq tf(x) + (1-t)f(y).$$

Definition A.2. (*Strongly convex function*) A function $f : \mathbb{R}^d \rightarrow \mathbb{R}$ is μ -strongly convex for some $\mu > 0$ if for any $x, y \in \mathbb{R}^d$ and $t \in (0, 1)$ we have

$$f(tx + (1-t)y) + \mu \frac{t(1-t)}{2} \|x - y\|^2 \leq tf(x) + (1-t)f(y).$$

Definition A.3. (*Polyak–Łojasiewicz function*) A differentiable function $f : \mathbb{R}^d \rightarrow \mathbb{R}$ is μ -Polyak–Łojasiewicz (PL) for some $\mu > 0$ if it is bounded from below and for any $x \in \mathbb{R}^d$ we have

$$f(x) - \inf f \leq \frac{1}{2\mu} \|\nabla f(x)\|^2.$$

B AutoGD proofs

Proof of Proposition 4.1. The sequence $(f(x_t))_{t \geq 0}$ satisfies $f(x_{t+1}) \leq f(x_t)$ for all t , and $f(x_t) \geq 0$. By the monotone convergence theorem, $\lim_{t \rightarrow \infty} f(x_t) = \underline{f} \geq 0$. $\square$

Proof of Proposition 4.2. We show $|x_{t+1}| > c^{-1}|x_t|$ and $\gamma_t = c^t \gamma_0$ for all $t \geq 0$. We proceed by induction. By assumption, we have that

$$|x_0| > \max \left\{ 1, \frac{1}{2pc\gamma_0}, \frac{1}{2pc\gamma_0} \cdot \frac{c+1}{c} \right\}.$$

Then note that

$$x_1 = x_0 - \gamma \nabla f(x_0) = x_0 \left(1 - 2p\gamma x_0^{2(p-1)} \right)$$

for some $\gamma \in \{c\gamma_0, \gamma_0, C\gamma_0\}$. Therefore, for any $\gamma \in \{c\gamma_0, \gamma_0, C\gamma_0\}$,

$$|x_1| = |x_0| \cdot \left(2p\gamma x_0^{2(p-1)} - 1 \right) \geq |x_0| \cdot (2p\gamma |x_0| - 1) > |x_0| \left(\frac{c+1}{c} - 1 \right) = c^{-1}|x_0|.$$

Therefore, we also have $\gamma_1 = c\gamma_0$. This concludes the base case.

Consider now any $t \geq 1$ and suppose that $|x_i| > c^{-1}|x_{i-1}|$ and that $\gamma_i = c^i \gamma_0$ for all $0 \leq i \leq t$. This implies that $|x_i| > c^{-i}|x_0|$ for any $i \leq t$. Therefore, for any $\gamma \in \{c\gamma_t, \gamma_t, C\gamma_t\}$,

$$\begin{aligned} 2p\gamma x_t^{2(p-1)} &\geq 2pc^{t+1}\gamma_0 x_t^{2(p-1)} \\ &\geq 2pc^{t+1}\gamma_0 |x_0|^{2(p-1)} c^{-2t(p-1)} \\ &= 2pc\gamma_0 |x_0| \cdot c^t |x_0|^{2p-3} c^{-2t(p-1)} \\ &> \frac{c+1}{c} \cdot c^t c^{-2t(p-1)} \\ &> \frac{c+1}{c}, \end{aligned}$$

and so

$$|x_{t+1}| = |x_t| \cdot \left(2p\gamma x_t^{2(p-1)} - 1 \right) > c^{-1}|x_t|,$$

and $\gamma_{t+1} = c\gamma_t = c^{t+1}\gamma_0$. This completes the proof. $\square$

In the following two sections we present two different proofs of convergence for AutoGD.

B.1 First proof of AutoGD convergence

Proof of Theorem 4.5. Since f is μ -PL, if $\|\nabla f(x)\| = 0$, we have $f(x) = 0$ and the bound trivially holds. It therefore suffices to check the bound for sequences x_t where $\|\nabla f(x_t)\| \neq 0$.

By the Taylor remainder theorem, for any $x, x' \in \mathbb{R}^d$,

$$f(x') = f(x) + \nabla f(x)^T(x' - x) + (x' - x)^T \int_0^1 (1-t) \nabla^2 f((1-t)x + tx') dt (x' - x).$$

If $x' = x - \gamma \nabla f(x)$ for some $\gamma > 0$, then

$$f(x') = f(x) - \gamma \|\nabla f(x)\|^2 + \gamma^2 \nabla f(x)^T \int_0^1 (1-t) \nabla^2 f(x - t\gamma \nabla f(x)) dt \nabla f(x),$$

and so if $\|\nabla f(x)\| \neq 0$,

$$f(x') = f(x) - \|\nabla f(x)\|^2 \left(\gamma - \frac{\gamma^2}{2} G(x, \gamma) \right).$$

Note that by smoothness,

$$f(x') \leq f(x) - \|\nabla f(x)\|^2 \left(\gamma - \frac{\gamma^2}{2} L \right). \quad (3)$$

Therefore, for $\gamma = 1/L$,

$$f(x') \leq f(x) - \frac{1}{2L} \|\nabla f(x)\|^2.$$

By smoothness again,

$$f(x') \geq f(x) - \|\nabla f(x)\|^2 \left(\gamma + \frac{\gamma^2}{2} L \right).$$

Therefore, for $\gamma \leq (-1 + \sqrt{2})/L$,

$$\begin{aligned} f(x') &\geq f(x) - \frac{1}{L} \|\nabla f(x)\|^2 \left(-1 + \sqrt{2} + \frac{(-1 + \sqrt{2})^2}{2} \right) \\ &\geq f(x) - \frac{1}{2L} \|\nabla f(x)\|^2. \end{aligned}$$

By Assumption 4.4, there exists an $\bar{\gamma} > 1/L$ such that

$$\bar{\gamma} - \frac{\bar{\gamma}^2}{2} G(x, \bar{\gamma}) \leq \frac{1}{2L},$$

and hence, for $x' = x - \bar{\gamma} \nabla f(x)$,

$$f(x') \geq f(x) - \frac{1}{2L} \|\nabla f(x)\|^2.$$

Now, by unimodality (Assumption 4.3), we have that the previous inequality holds for any $\gamma \geq \bar{\gamma}$. Therefore, for all x , the position-dependent optimal learning rate $\gamma^*(x)$ satisfies

$$\gamma^*(x) \in \left[\frac{\sqrt{2}-1}{L}, \bar{\gamma} \right], \quad f(x - \gamma^*(x) \nabla f(x)) \leq f(x) - \frac{1}{2L} \|\nabla f(x)\|^2.$$

We first examine the time that it takes for the learning rate to reach a desirable set $\left(C^{-1} \frac{\sqrt{2}-1}{L}, c^{-1} \bar{\gamma} \right)$.

Suppose $\gamma_0 \geq c^{-1} \bar{\gamma}$. Then by Assumption 4.3, the learning rate must initially shrink for k iterations, until $\gamma < c^{-1} \bar{\gamma}$, where

$$k \leq \left\lfloor \log_{c^{-1}} \frac{\gamma_0}{c^{-1} \bar{\gamma}} \right\rfloor + 1 = \left\lfloor \log_{c^{-1}} \frac{\gamma_0}{\bar{\gamma}} - 1 \right\rfloor + 1 \leq \log_{c^{-1}} \frac{\gamma_0}{\bar{\gamma}}.$$

Similarly, if $\gamma_0 \leq C^{-1} \frac{\sqrt{2}-1}{L}$, the learning rate must initially grow for k iterations until $\gamma > C^{-1} \frac{\sqrt{2}-1}{L}$, where

$$k \leq \left\lceil \log_C \frac{C^{-1}(\sqrt{2}-1)}{L\gamma_0} \right\rceil + 1 = \left\lceil \log_C \frac{(\sqrt{2}-1)}{L\gamma_0} - 1 \right\rceil + 1 \leq \log_C \frac{(\sqrt{2}-1)}{L\gamma_0}.$$

Therefore, for the first $k \leq t_0$ iterations, where

$$t_0 = \max \left\{ 0, \log_{c^{-1}} \frac{\gamma_0}{\bar{\gamma}}, \log_C \frac{(\sqrt{2}-1)}{L\gamma_0} \right\},$$

we have $f(x_{t+1}) \leq f(x_t)$. We then also have $\gamma_k \in (C^{-1} \frac{\sqrt{2}-1}{L}, c^{-1}\bar{\gamma})$.

For the remainder of the proof, we therefore assume $\gamma_t \in (C^{-1} \frac{\sqrt{2}-1}{L}, c^{-1}\bar{\gamma})$. By Assumption 4.3, since $\gamma^*(x_t) \in [\frac{\sqrt{2}-1}{L}, \bar{\gamma}]$, the only case in which it is possible for the learning rate to decrease is if $\gamma_t > \gamma^*(x_t) \geq \frac{\sqrt{2}-1}{L}$. Similarly, the only case in which it is possible for the learning to increase is if $\gamma_t < \gamma^*(x_t) \leq \bar{\gamma}$. Therefore,

$$\min\{c, C^{-1}\} \frac{\sqrt{2}-1}{L} < \gamma_{t+1} < \max\{C, c^{-1}\} \bar{\gamma}.$$

If $\gamma_{t+1} > c^{-1}\bar{\gamma}$ (which is only possible if $C > c^{-1}$), then the learning rate will decrease for

$$k \leq \left\lceil \log_{c^{-1}} \frac{C}{c^{-1}} \right\rceil$$

iterations until $\gamma_t < c^{-1}\bar{\gamma}$ again. Likewise, if $\gamma_{t+1} < C^{-1} \frac{\sqrt{2}-1}{L}$ (which is again only possible if $C > c^{-1}$), the learning rate will increase for

$$k \leq \left\lceil \log_C \frac{C^{-1}}{c} \right\rceil$$

iterations until $\gamma_t > C^{-1} \frac{\sqrt{2}-1}{L}$. Also, if $\gamma_t > c^{-1}\gamma^*(x_t)$, then $\gamma_{t+1} = c\gamma_t$ and $f(x_{t+1}) \leq f(x_t)$. Note that the maximum number of iterations where this can occur in a row (starting from iteration t) is

$$k \leq \left\lceil \log_{c^{-1}} \frac{\gamma_t}{\frac{\sqrt{2}-1}{L}} \right\rceil = \left\lceil \log_{c^{-1}} \frac{L\gamma_t}{\sqrt{2}-1} \right\rceil \leq \left\lceil \log_{c^{-1}} \frac{Lc^{-1}\bar{\gamma}}{\sqrt{2}-1} \right\rceil.$$

Finally, suppose $\gamma_t \leq c^{-1}\gamma^*(x_t)$. Therefore, $c\gamma_t \leq \gamma^*(x_t)$. By Assumption 4.3, the function is decreasing in γ until γ^* ; therefore, we can bound the descent by assuming we pick the lower learning rate $c\gamma_t$, and assume γ_t takes its smallest value $\gamma_t = \frac{\sqrt{2}-1}{CL}$ (although it may actually be the case that we pick $C\gamma_t$ or γ_t). From Eq. (3), and the fact that f is μ -PL,

$$\begin{aligned} f(x_{t+1}) &\leq f(x_t) - \|\nabla f(x)\|^2 \left(c \frac{\sqrt{2}-1}{CL} - \left(c \frac{\sqrt{2}-1}{CL} \right)^2 \frac{L}{2} \right) \\ &= f(x_t) - \frac{c(\sqrt{2}-1)}{2C^2L} (2C - c(\sqrt{2}-1)) \|\nabla f(x)\|^2 \\ &\leq f(x_t) \left(1 - \frac{\mu c(\sqrt{2}-1)(2C - c(\sqrt{2}-1))}{C^2L} \right). \end{aligned}$$

Therefore, from entry in $(C^{-1} \frac{\sqrt{2}-1}{L}, c^{-1}\bar{\gamma})$, we may perform

$$k \leq \left\lceil \log_{c^{-1}} \frac{Lc^{-1}\bar{\gamma}}{\sqrt{2}-1} \right\rceil + \max \left\{ \left\lceil \log_C \frac{C^{-1}}{c} \right\rceil, \left\lceil \log_{c^{-1}} \frac{C}{c^{-1}} \right\rceil \right\} + 1$$

iterations until reentry into the set, of which at least one iteration guarantees strict descent. Combining the above bounds yields the final result. $\square$

B.2 Second proof of AutoGD convergence

An alternate proof of the convergence of AutoGD, using slightly different assumptions and a different proof technique, is presented below. We assume that we shrink by a factor of c if we detect a function increase at the smallest learning rate in the grid.

Assumption B.1. For all $x \in \mathbb{R}^d$ with $\|\nabla f(x)\| \neq 0$, there exists $\bar{\gamma}(x)$ such that $f(x - \gamma \nabla f(x)) > f(x)$ if and only if $\gamma > \bar{\gamma}(x)$.

For L -smooth f , define $\underline{\gamma} = \frac{2(C-1)}{LC^2}$. Recall the definition of G given by Eq. (2), and that for convenience we assume (without loss of generality) that $\min_x f(x) = 0$.

Assumption B.2. There exists $\epsilon \in (0, 1]$ such that for all $x \in \mathbb{R}^d$ with $\|\nabla f(x)\| \neq 0$ and $\gamma \in [\underline{\gamma}, \bar{\gamma}(x)]$, we have $c\gamma G(x, c\gamma) \leq 2(1 - \epsilon)$.

Assumption B.3. There exists $\underline{G} > 0$ such that for all $x \in \mathbb{R}^d$ with $\|\nabla f(x)\| \neq 0$, we have $G(x, \bar{\gamma}(x)) \geq \underline{G}$.

Assumptions B.2 and B.3 are technical, but are satisfied for reasonable quadratic functions. In particular, consider an arbitrary quadratic function $f(x) = x^T A x$ where A is positive definite. The condition number of this function is $\kappa = \lambda_{\max}(A)/\lambda_{\min}(A)$, where $\lambda_{\min}, \lambda_{\max}$ are the minimum and maximum eigenvalues. In this case, $L = \lambda_{\max}(A)$ and $\lambda_{\min}(A) = \mu$. We show that in this case, Assumption B.1 holds with $\bar{\gamma} = x^T A^2 x / (x^T A^3 x)$. Also, Assumption B.2 holds with any $\epsilon \leq 1 - 2c$, which is independent of κ , and Assumption B.3 holds with $\underline{G} = 4\lambda_{\min}(A) = 4\mu$. Therefore, the main restriction is that $c < 1/2$, to allow for $\epsilon > 0$.

To see this, note that

$$f(x - \gamma \nabla f(x)) = x^T (I - 2\gamma A)^T A (I - 2\gamma A) x = x^T (I - 2\gamma A) A (I - 2\gamma A) x,$$

and so, for $x \neq 0$, we have $f(x - \gamma \nabla f(x)) - f(x) > 0$ if and only if $\gamma > \bar{\gamma}(x)$, where

$$\bar{\gamma}(x) = \frac{x^T A^2 x}{x^T A^3 x}.$$

Now, because $\nabla f(x) = 2Ax$ and $\nabla^2 f = 2A$, we have, for $x \neq 0$,

$$G(x, \gamma) = \frac{16x^T A^3 x}{4x^T A^2 x} = \frac{4x^T A^3 x}{x^T A^2 x} = \frac{4}{\bar{\gamma}(x)}.$$

Therefore, for any $x \in \mathbb{R}^d$ ($x \neq 0$) and $\gamma \in [\underline{\gamma}, \bar{\gamma}(x)]$, we have

$$c\gamma G(x, c\gamma) \leq c\bar{\gamma}(x) \cdot \frac{4}{\bar{\gamma}(x)} = 4c.$$

Thus, condition Assumption B.2 is guaranteed to hold if $\epsilon \leq 1 - 2c$. Further, for Assumption B.3, note that

$$\inf_{x \in \mathbb{R}^d} G(x, \bar{\gamma}(x)) = \inf_{x \in \mathbb{R}^d} \frac{4x^T A^3 x}{x^T A^2 x} = \inf_{y \in \mathbb{R}^d: \|y\|=1} 4y^T A y \geq 4\lambda_{\min}(A) = 4\mu.$$

Theorem B.4. Suppose f is L -smooth, μ -Polyak-Łojasiewicz, twice continuously differentiable, and that Assumptions B.1 to B.3 hold. Let

$$\begin{aligned} \tau &= \left\lceil \log_{c^{-1}} \frac{LC}{\underline{G}} \right\rceil + \lceil -\log_C(c) \rceil + 1 \\ t_0 &= \max \left\{ 0, \left\lceil \log_C \frac{2(C-1)}{LC^2\gamma_0} \right\rceil, \left\lceil \log_{c^{-1}} \frac{L\gamma_0}{2} \right\rceil + \max\{0, \lceil -1 - \log_C(c) \rceil\} \right\} \end{aligned}$$

Then

$$f(x_t) \leq f(x_0)(1 - 2c\underline{\gamma}\epsilon\mu)^{\lfloor \frac{t-t_0}{\tau} \rfloor}.$$

Proof of Theorem B.4. As before, since f is μ -PL, if $\|\nabla f(x)\| = 0$, we have $f(x) = 0$ and the bound trivially holds. It therefore suffices to check the bound for sequences x_t where $\|\nabla f(x_t)\| \neq 0$.

By the Taylor remainder theorem, for any $x, x' \in \mathbb{R}^d$,

$$f(x') = f(x) + \nabla f(x)^T(x' - x) + (x' - x)^T \int_0^1 (1-t) \nabla^2 f((1-t)x + tx') dt (x' - x).$$

If $x' = x - \gamma \nabla f(x)$ for some $\gamma > 0$, then

$$f(x') = f(x) - \gamma \|\nabla f(x)\|^2 + \gamma^2 \nabla f(x)^T \int_0^1 (1-t) \nabla^2 f(x - t\gamma \nabla f(x)) dt \nabla f(x),$$

and so

$$f(x') = f(x) - \|\nabla f(x)\|^2 \left(\gamma - \frac{\gamma^2}{2} G(x, \gamma) \right). \quad (4)$$

Suppose $\|\nabla f(x)\| \neq 0$. Note that by continuity of f and Assumption B.1, we have

$$f(x - \bar{\gamma}(x) \nabla f(x)) = f(x).$$

From Eq. (4), this must mean that if $\|\nabla f(x)\| \neq 0$ and $\bar{\gamma}(x) \neq 0$, we have

$$\bar{\gamma}(x) = \frac{2}{G(x, \bar{\gamma})}. \quad (5)$$

This property will come in useful in the remainder of the proof. In fact, at any point where $\|\nabla f(x)\| \neq 0$, it must necessarily also be the case that $\bar{\gamma}(x) \neq 0$ by differentiability of f . Therefore, it suffices to check that $\|\nabla f(x)\| \neq 0$ in order to use Eq. (5). Since $\bar{\gamma}(x)$ is undefined when $\|\nabla f(x)\| = 0$, in what follows we set $\bar{\gamma}(x) = \infty$, forcing Assumption B.1 to hold in this case as well.

Suppose the state of the optimizer is x_t with learning rate γ_t . Denote the three potential next states

$$\bar{x}_{t+1} = x_t - C\gamma_t \nabla f(x_t) \quad \tilde{x}_{t+1} = x_t - \gamma_t \nabla f(x_t) \quad \underline{x}_{t+1} = x_t - c\gamma_t \nabla f(x_t).$$

We split the proof into three cases: γ_t too large ($\gamma_t > \bar{\gamma}(x_t)$), γ_t too small ($\gamma_t < \underline{\gamma}$), and γ_t within bounds. If γ_t is too small, we show that $\gamma_{t+1} = C\gamma_t$ and $f(x_{t+1}) \leq f(x_t)$. If γ_t is too large, we show that $\gamma_{t+1} = c\gamma_t$ and $f(x_{t+1}) \leq f(x_t)$. If γ_t is within bounds, we show that $f(x_{t+1}) \leq af(x_t)$ for some $a \in [0, 1]$, and that γ_{t+1} is not too far from being within bounds.

γ_t too small ($\gamma_t < \underline{\gamma}$): Note that by L -smoothness, $\sup_{x, \gamma} |G(x, \gamma)| \leq L$. Substituting the three learning rates into Eq. (4),

$$\begin{aligned} f(\bar{x}_{t+1}) &\leq f(x_t) - \|\nabla f(x_t)\|^2 \left(C\gamma - \frac{C^2\gamma^2 L}{2} \right) \\ f(\tilde{x}_{t+1}) &\geq f(x_t) - \gamma \|\nabla f(x_t)\|^2 \\ f(\underline{x}_{t+1}) &\geq f(x_t) - c\gamma \|\nabla f(x_t)\|^2. \end{aligned}$$

Therefore, the minimum is guaranteed to occur at $\bar{x}_{t+1}$ with learning rate $C\gamma_t$ if

$$C - \frac{C^2\gamma L}{2} > 1 \iff \gamma < \underline{\gamma} = \frac{2(C-1)}{LC^2}.$$

In this case, using the fact that f is μ -PL,

$$\begin{aligned} f(x_{t+1}) &= f(\bar{x}_{t+1}) \\ &\leq f(x_t) - \|\nabla f(x_t)\|^2 \left(C\gamma - \frac{C^2\gamma^2 L}{2} \right) \\ &\leq f(x_t) \left(1 - 2\mu \left(C\gamma - \frac{C^2\gamma^2 L}{2} \right) \right). \end{aligned}$$

Note that if $\gamma_t < \underline{\gamma}$, using the relation from Eq. (5), we have

$$\gamma_{t+1} = C\gamma_t < C\underline{\gamma} = \frac{2C(C-1)}{LC^2} \leq \frac{2}{L} \leq \inf_{\gamma > 0} \frac{2}{G(x_{t+1}, \gamma)} \leq \bar{\gamma}(x_{t+1}),$$

and hence γ_{t+1} will never exceed $\bar{\gamma}(x_{t+1})$ when γ_t increases from below $\underline{\gamma}$.

It takes

$$k \leq \left\lceil \log_C \frac{\gamma}{\gamma_t} \right\rceil = \left\lceil \log_C \frac{2(C-1)}{LC^2\gamma_t} \right\rceil$$

iterations until $C^k\gamma_t \geq \underline{\gamma}$.

γ_t **too large** ($\gamma_t > \bar{\gamma}(x_t)$): By assumption, we have that $f(x - \gamma\nabla f(x)) > f(x)$ iff $\gamma > \bar{\gamma}(x)$. Therefore if $\gamma_t > \bar{\gamma}(x_t)$, then $C\gamma_t > \bar{\gamma}(x_t)$ as well. There are two cases to consider: $c\gamma_t > \bar{\gamma}(x_t)$ and $c\gamma_t \leq \bar{\gamma}(x_t)$. If $c\gamma_t > \bar{\gamma}(x_t)$, then all three choices increase the objective, and so the algorithm stands still and shrinks the learning rate: $x_{t+1} = x_t$ and $\gamma_{t+1} = c\gamma_t$. If $c\gamma_t \leq \bar{\gamma}(x_t)$, then the minimum occurs at $\underline{x}_{t+1}$, and $f(x_{t+1}) \leq f(x_t)$ with $\gamma_{t+1} = c\gamma_t$.

Since $G(x, \bar{\gamma}(x)) \leq L$, and either $\bar{\gamma}(x) = \frac{2}{G(x, \bar{\gamma}(x))}$ or $\bar{\gamma}(x) = \infty$, we have that $\bar{\gamma}(x) \geq 2/L$. Therefore, it takes

$$k \leq \left\lceil \log_{c^{-1}} \frac{L\gamma_t}{2} \right\rceil$$

iterations until $c^k\gamma_t \leq \bar{\gamma}(x_{t+k})$. Note that even if $c^{k-1}\gamma_t > \bar{\gamma}(x_{t+k-1})$, it is still possible to have $c^k\gamma_t < \gamma$, in which case we need to increase the learning rate before $\gamma_t \in [\underline{\gamma}, \bar{\gamma}(x)]$. Since $\bar{\gamma}(x) \geq 2/L$, this takes

$$k \leq \max\left\{0, \left\lceil \log_C \frac{\gamma}{2c/L} \right\rceil\right\} = \max\left\{0, \left\lceil \log_C \frac{(C-1)}{cC^2} \right\rceil\right\} \leq \max\{0, \lceil -1 - \log_C(c) \rceil\}.$$

γ_t **within bounds** ($\underline{\gamma} \leq \gamma_t \leq \bar{\gamma}(x_t)$): Suppose $\gamma_t \in [\underline{\gamma}, \bar{\gamma}(x_t)]$. By Eq. (4),

$$f(x_{t+1}) \leq f(\underline{x}_{t+1}) = f(x_t) - c\gamma_t \|\nabla f(x_t)\|^2 \left(1 - \frac{c\gamma_t G(x_t, c\gamma_t)}{2}\right).$$

By Assumption B.2, and the fact that f is μ -PL, if $\|\nabla f(x)\| \neq 0$,

$$\begin{aligned} f(x_{t+1}) &\leq f(x_t) - c\gamma_t \epsilon \|\nabla f(x_t)\|^2 \\ &\leq f(x_t) - c\underline{\gamma} \epsilon \|\nabla f(x_t)\|^2 \\ &\leq f(x_t) - c\underline{\gamma} \epsilon 2\mu f(x_t) \\ &= f(x_t)(1 - 2c\underline{\gamma} \epsilon \mu). \end{aligned}$$

That is, $f(x_{t+1}) \leq af(x_t)$ with $0 \leq a = 1 - 2c\underline{\gamma} \epsilon \mu < 1$. (We know $a \geq 0$ when f is not identically equal to zero everywhere, or else $\min_x f(x) < 0$, for a contradiction.) Otherwise, if $\|\nabla f(x)\| = 0$, since f is μ -PL, we know that $f(x_{t+1}) = f(x_t) = 0$ and so $f(x_{t+1}) \leq af(x_t)$ in this case, as well. Further, note that

$$\gamma_{t+1} \geq c\gamma_t \geq \underline{\gamma}$$

and

$$\gamma_{t+1} \leq C\gamma_t \leq C\bar{\gamma}(x_t) = \frac{2C}{G(x_t, \bar{\gamma}(x_t))} \leq \frac{2C}{\underline{G}}.$$

Therefore, using the earlier bounds on the number of iterations in the too large and too small cases, it takes

$$\begin{aligned} k &\leq \max\left\{\left\lceil \log_C \frac{2(C-1)}{LC^2c\underline{\gamma}} \right\rceil, \left\lceil \log_{c^{-1}} \frac{2LC}{2\underline{G}} \right\rceil + \max\{0, \lceil -1 - \log_C(c) \rceil\}\right\} \\ &= \max\left\{\lceil -\log_C(c) \rceil, \left\lceil \log_{c^{-1}} \frac{LC}{\underline{G}} \right\rceil + \max\{0, \lceil -1 - \log_C(c) \rceil\}\right\} \\ &= \left\lceil \log_{c^{-1}} \frac{LC}{\underline{G}} \right\rceil + \lceil -\log_C(c) \rceil \end{aligned}$$

iterations to return to $\gamma_t \in [\underline{\gamma}, \bar{\gamma}(x_t)]$. Combining the behaviour in the three regions yields the stated result. $\square$

C AutoSGD implementation details and theory

C.1 Online test statistic calculation and decision process

Within episode t , the sequence of test statistics $\underline{Z}_{t,k}, Z_{t,k}, \bar{Z}_{t,k}$ can be computed online without storing the traces $(x_{t,k})_{k=0}^{T_t}$. Recall that

$$Z_{t,k} = \frac{\sum_{j=0}^k \frac{1}{2} (\Delta_{t,j}^{(1)} + \Delta_{t,j}^{(2)})}{\sqrt{\sum_{j=0}^k \max\left(\epsilon, \frac{1}{2} (\Delta_{t,j}^{(1)} - \Delta_{t,j}^{(2)})^2\right)}}.$$

It suffices to store only two floating point numbers for $\sum_{j=0}^k \frac{1}{2} (\Delta_{t,j}^{(1)} + \Delta_{t,j}^{(2)})$ and $\sum_{j=0}^k \max\left(\epsilon, \frac{1}{2} (\Delta_{t,j}^{(1)} - \Delta_{t,j}^{(2)})^2\right)$. These two quantities can be updated online with rolling sums. Our proposed AutoSGD decision process is given in Algorithm 2. For the description of the decision algorithm, we define $Z_{t,k}^{(\gamma)}$ to be the statistic among $\underline{Z}_{t,k}, Z_{t,k}, \bar{Z}_{t,k}$ corresponding to the stream with learning rate γ .

Algorithm 2 decision()

Require: Streams $(x_{t,j})_{j=0}^k, (\bar{x}_{t,j})_{j=0}^k$ with learning rates in $E_t := \{c\gamma_t, \gamma_t, C\gamma_t\}$, statistic threshold z^* (default: 1.96), minimum number of samples M (default: 30).

```

1: if  $k + 1 < M$  then                                     ▷ not enough samples
2:    $I_t, D_t, S_t, R_t \leftarrow 0$ 
3: else
4:    $A \leftarrow \{\gamma \in E_t : Z_{t,k}^{(\gamma)} > z^*, Z_{t,k}^{(\gamma')} \geq -z^* \text{ for any } \gamma' < \gamma, \gamma' \in E_t\}$ 
5:   if  $A \neq \emptyset$  then                                     ▷ significant objective function reduction
6:      $\gamma' \leftarrow \max A$ 
7:     if  $\gamma' = c\gamma_t$  then                                   ▷ decrease the learning rate
8:        $I_t, D_t, S_t, R_t \leftarrow 0, 1, 0, 0$ 
9:     else if  $\gamma' = \gamma_t$  then                               ▷ stay at the current learning rate
10:       $I_t, D_t, S_t, R_t \leftarrow 0, 0, 1, 0$ 
11:    else                                                  ▷ increase the learning rate
12:       $I_t, D_t, S_t, R_t \leftarrow 1, 0, 0, 0$ 
13:    end if
14:  else
15:     $A \leftarrow \{\gamma \in \{c\gamma_t, \gamma_t\} : Z_{t,k}^{(\gamma)} \geq -z^*, Z_{t,k}^{(\gamma')} \geq -z^* \text{ for any } \gamma' < \gamma, \gamma' \in \{c\gamma_t, \gamma_t\}\}$ 
16:    if  $A \neq \emptyset$  then
17:       $\gamma' \leftarrow \max A$ 
18:      if  $\gamma' = c\gamma_t$  then                                   ▷ decrease the learning rate
19:         $I_t, D_t, S_t, R_t \leftarrow 0, 1, 0, 0$ 
20:      else                                               ▷ stay at the current learning rate
21:         $I_t, D_t, S_t, R_t \leftarrow 0, 0, 1, 0$ 
22:      end if
23:    else                                               ▷ evidence of objective increase at all learning rates
24:       $I_t, D_t, S_t, R_t \leftarrow 0, 0, 0, 1$ 
25:    end if
26:  end if
27: end if
28: return  $I_t, D_t, S_t, R_t$ 

```

C.2 Stochastic convergence theory

Theorem C.12 presents the main AutoSGD convergence result and its proof. Note that Theorem C.12 is just a more specific version of Theorem 4.6 from the main text with all the constants and value constraints specified.

The proof of convergence of AutoSGD relies on the following high-level strategy with 3 steps.

- Step 1: We first consider the “not bad *subsequence*” of episode iterations x_t, γ_t such that $a\gamma_t \leq f(x_t)$ (for some proportionality constant $a > 0$). At these episodes, the step size is small enough that the inner SGD streams should result in an expected descent $\mathbb{E}[f(x_{t+1})|x_t, \gamma_t] \leq f(x_t)$ (though it may be a vanishingly small amount of descent). In between these iterations, the algorithm can take excursions into the “bad” region where $a\gamma_t > f(x_t)$ (the step size is too large) for some random number of episodes. Lemma C.1 shows that these excursions are not *too* bad, and only result in at most a small increase after the initial one-step descent $\mathbb{E}[f(x_{t+1})|x_t, \gamma_t] \leq f(x_t)$.
- Step 2: We next consider the “good *subsubsequence*” of episode iterations x_t, γ_t such that $\ell f(x_t) \leq a\gamma_t \leq f(x_t)$ (for some other constant $\ell > 0$). In these episodes, the step size is both small enough to guarantee descent, and large enough to guarantee *non-negligible* descent. Lemma C.2 uses a stochastic Lyapunov function technique (i.e., a drift function, in the terminology of the MCMC literature [DMPS18]) to bound the amount of time on excursions away from this set.
- Step 3: Finally, Theorem C.3 puts these two results together. If we assume that the one-step descent in the “not bad” region where $a\gamma_t \leq f(x_t)$ satisfies $\mathbb{E}[f(x_{t+1})|x_t, \gamma_t] \leq (1 - \eta \frac{a\gamma_t}{f(x_t)})f(x_t)$ (i.e., vanishing descent as $\gamma_t \rightarrow 0$), then the overall descent on the “not bad subsequence” of episodes forms a nonnegative supermartingale. At that point, the optional stopping theorem shows that the “good subsubsequence” exhibits geometric descent, and the drift function bounds from Lemma C.2 guarantee that the good subsubsequence occurs regularly in the original episode sequence. The remainder of this section after presenting Lemmas C.1 and C.2 and Theorem C.3 is devoted to posing specific assumptions about the decision process (that determines $I_t, S_t, D_t, R_t, \tau_t$) to enable verification of the conditions of these three abstract results for AutoSGD, resulting in Theorem C.12.

Note that the proofs of all results in this section are admittedly rather technical and not very illuminating; to clarify the exposition, we will simply present and discuss results here, and defer the proofs to Appendix C.3.

We begin with Lemma C.1, which analyzes the expected change in f on the “not bad subsequence” in between excursions to the “bad” region. Here a is the aforementioned proportionality constant for the “not bad” region. The constant $b > 0$ characterizes the amount of increase of f during excursions into the “bad” region; in AutoSGD, this will be proportional to the probability of R_t not triggering in the bad region (which should be very small). The proportionality to γ_t in terms with b is crucial for later showing that the “not bad subsequence” forms a nonnegative supermartingale; in particular, the closer γ_t is to 0 (the farther from the bad region it is), the less increase we expect, because the episode sequence is most likely to simply stay in the “not bad” region. The constant $v \in [0, 1)$ characterizes the expected decrease of the step size while in the bad region; in AutoSGD, v should be just slightly greater than $c < 1$, since R_t is very likely in that region (at which point $\gamma_{t+1} \leftarrow c\gamma_t$).

Lemma C.1. *Consider a discrete-time Markov chain (x_t, γ_t) , $t \in \mathbb{N}_0$ on $\mathbb{R}^{d+1}$ and a nonnegative function $f : \mathbb{R}^d \rightarrow \mathbb{R}$, $f \geq 0$. Suppose there exist constants $a, b, C \geq 0$ and $v \in [0, 1)$ such that for all $t \in \mathbb{N}_0$, we have $\gamma_{t+1} \leq C\gamma_t$ a.s. and*

$$a\gamma_t > f(x_t) \implies \mathbb{E}[\gamma_{t+1}|x_t, \gamma_t] \leq v\gamma_t \quad \text{and} \quad \mathbb{E}[f(x_{t+1})|x_t, \gamma_t] \leq f(x_t) + b\gamma_t.$$

Fix a particular $t \in \mathbb{N}_0$, and let $\tau \in \mathbb{N}$ be the smallest index such that $\tau > t$ and $a\gamma_\tau \leq f(x_\tau)$, or ∞ if no such index exists. Then

$$\mathbb{E}[f(x_\tau)|x_t, \gamma_t] \leq \mathbb{E}[f(x_{t+1})|x_t, \gamma_t] + \gamma_t \frac{Cb}{1-v},$$

where $f(x_\tau) = \lim_{k \rightarrow \infty} f(x_{\tau \wedge k})$ and $\mathbb{E}[f(x_\tau)|x_t, \gamma_t] = \lim_{k \rightarrow \infty} \mathbb{E}[f(x_{\tau \wedge k})|x_t, \gamma_t]$ are well-defined random variables.

Next, given an appropriate drift function V , Lemma C.2 characterizes the amount of time spent away from the region where $V \leq 1$. Note that Lemma C.2 is not novel, and similar results appear commonly in the MCMC convergence literature in the analysis of geometric ergodicity [DMPS18]. Later in Theorem C.3, we will assume that V is designed such that the region where $V \leq 1$ corresponds to the region where $\ell f(x_t) \leq a\gamma_t \leq f(x_t)$ (i.e., the “good” region where sufficient decrease occurs). In

AutoSGD, the constant ξ will be related to the probability of R_t in the “bad” region $a\gamma_t > f(x_t)$, and the probability of I_t when γ_t is far too small; in other words, how likely the algorithm is to shrink the step size when it is too large, and how likely it is to grow it when it is too small. The constant ζ can take any finite value, and is just a technical constant in the application to AutoSGD below.

Lemma C.2. *Given a discrete-time Markov chain $x_t, \gamma_t, t \in \mathbb{N}_0$ on $\mathbb{R}^{d+1}$, suppose there exist constants $\zeta \geq 0, \xi \in [0, 1)$ and a measurable function $V : \mathbb{R}^{d+1} \rightarrow \mathbb{R}$ such that*

$$\mathbb{E}[V(x_{t+1}, \gamma_{t+1}) | x_t, \gamma_t] \leq \xi V(x_t, \gamma_t) + \zeta \mathbb{1}[V(x_t, \gamma_t) \leq 1] \quad a.s.$$

Fix any $t \in \mathbb{N}_0$, and let $\tau \in \mathbb{N}$ be the smallest index such that $\tau > t$ and $V(x_\tau, \gamma_\tau) \leq 1$. Then

$$\forall k \in \mathbb{N}, \quad \mathbb{P}(\tau > k | x_t, \gamma_t) \leq \xi^{k-1} (\xi V(x_t, \gamma_t) + \zeta \mathbb{1}[V(x_t, \gamma_t) \leq 1]) \quad a.s.$$

Finally, Theorem C.3 makes some slight additional assumptions on the one-step descent of f and the design of the drift function V to put the two above lemmas together for the main abstract result about convergence. Here ℓ is the earlier-mentioned constant for the lower bound on the “good” region. We will see below (in Lemma C.9) that due to the use of SGD for inner iterations, $\mathbb{E}[f(x_{t+1}) | x_t, \gamma_t]$ is always a mixture of $f(x_t)$ and a quantity proportional to γ_t , and so the constant B captures that proportionality constant. Finally, η captures the one-step descent of f in the “not bad” region in a manner that vanishes as $\gamma_t \rightarrow 0$.

Theorem C.3. *Consider a discrete-time Markov chain $x_t, \gamma_t, t \in \mathbb{N}_0$ on $\mathbb{R}^{d+1}$, functions $f : \mathbb{R}^d \rightarrow \mathbb{R}$ and $V : \mathbb{R}^{d+1} \rightarrow \mathbb{R}$, and constants a, b, C, v, ζ, ξ all satisfying the conditions of Lemmas C.1 and C.2. Suppose additionally that there exist constants $B, \ell > 0, \eta \in (0, 1]$ such that*

$$\begin{aligned} \mathbb{E}[f(x_{t+1}) | x_t, \gamma_t] &\leq \max\{f(x_t), B\gamma_t\} \\ V(x_t, \gamma_t) \leq 1 &\implies \ell \leq \frac{a\gamma_t}{f(x_t)} \leq 1 \\ a\gamma_t \leq f(x_t) &\implies \mathbb{E}[f(x_{t+1}) | x_t, \gamma_t] \leq f(x_t) \left(1 - \eta \frac{a\gamma_t}{f(x_t)}\right) \\ \eta - \frac{Cb}{a(1-v)} &> 0. \end{aligned} \tag{6}$$

Let t_n be the n^{th} index t at which $V(x_t, \gamma_t) \leq 1$. Then

$$\forall n \in \mathbb{N}, \quad \mathbb{E}[f(x_{t_n})] \leq \nu^{n-1} f_0 \quad \text{and} \quad \mathbb{E}[t_n] \leq \frac{V_0}{1-\xi} + (n-1) \frac{\xi + \zeta}{1-\xi},$$

where

$$\begin{aligned} \nu &= \left(1 - \ell \left(\eta - \frac{Cb}{a(1-v)}\right)\right) < 1 \\ f_0 &= \max\{f(x_0), B\gamma_0\} + \gamma_0 \frac{Cb}{1-v} \quad V_0 = \xi V(x_0, \gamma_0) + \zeta \mathbb{1}[V(x_0, \gamma_0) \leq 1]. \end{aligned}$$

The remainder of this section is devoted to applying Lemmas C.1 and C.2 and Theorem C.3 to the special case of AutoSGD. We begin with technical assumptions Assumptions C.4 to C.8, using the notation and basic setup established in the main text. Note that while I_t, S_t, D_t, R_t are binary indicator variables for the various decision events in episode t , we use a slight abuse of notation below in that the indicators are treated also as events in probability statements; for example, $\mathbb{P}(I_t)$ is used as a shorthand for $\mathbb{P}(I_t = 1)$. We use these assumptions to prove Lemmas C.9 to C.11, and finally conclude the section with the proof of Theorem C.12 (the more detailed version of Theorem 4.6 from the main text), which amounts to translating the constants from Assumptions C.4 to C.8 into the constants needed for Lemmas C.1 and C.2 and Theorem C.3.

Note that the below set of assumptions and theoretical analysis are designed to illustrate the properties necessary for a *general* decision process to yield convergent AutoSGD. We do not attempt to apply these results to our specific recommended decision process from Appendix C.1 in the interest of avoiding significant amounts of uninteresting computation, although that decision process is designed to intuitively follow the below desiderata.

Assumption C.4 stipulates some basic properties of f . The technical condition in the display is satisfied in fairly general settings, e.g., when f has Lipschitz gradients and $g(x, u)$ has a variance bound proportional to $\|\nabla f(x)\|^2 + b$.

Assumption C.4. $f : \mathbb{R}^d \rightarrow \mathbb{R}$ is twice continuously differentiable, μ -PL, attains its minimum with value $\min_{x \in \mathbb{R}^d} f(x) = 0$, and there exist $\alpha, \beta \geq 0$ such that $\forall 0 \leq \gamma \leq \alpha^{-1}$,

$$\left| \mathbb{E} \left[g(x, u)^T \int_0^1 2(1-s) \nabla^2 f(x - s\gamma g(x, u)) ds g(x, u) \right] \right| \leq \alpha \|\nabla f(x)\|^2 + \beta, \quad u \sim \phi.$$

Assumption C.5 captures the desideratum that the decision process tends to prefer lower objective values. For example, in particular, it says that $I_t \mathbb{1}[\tau_t = k]$ is negatively correlated with $f(\bar{x}_{t,k})$. In other words, it says that on events where I_t triggers at inner iteration $\tau_t = k$, $f(\bar{x}_{t,k})$ tends to be smaller than it would be absent any decision process occurring. We expect that this assumption is satisfied by reasonable decision processes.

Assumption C.5. For all $k \in \mathbb{N}$,

$$\begin{aligned} \mathbb{E}[I_t \mathbb{1}[\tau_t = k] f(\bar{x}_{t,k}) | x_t, \gamma_t] &\leq \mathbb{P}(I_t, \tau_t = k | x_t, \gamma_t) \mathbb{E}[f(\bar{x}_{t,k}) | x_t, \gamma_t] \\ \mathbb{E}[S_t \mathbb{1}[\tau_t = k] f(x_{t,k}) | x_t, \gamma_t] &\leq \mathbb{P}(S_t, \tau_t = k | x_t, \gamma_t) \mathbb{E}[f(x_{t,k}) | x_t, \gamma_t] \\ \mathbb{E}[D_t \mathbb{1}[\tau_t = k] f(\underline{x}_{t,k}) | x_t, \gamma_t] &\leq \mathbb{P}(D_t, \tau_t = k | x_t, \gamma_t) \mathbb{E}[f(\underline{x}_{t,k}) | x_t, \gamma_t]. \end{aligned}$$

Assumption C.6 captures the need for the decision process to reset and decrease the step size when it is too large (R_t), and to increase the step size when it is too small (I_t). In particular, the decision process should have a very small probability δ of making the wrong decision in either case. It also guarantees that $\gamma_t \leq \alpha^{-1}$ a.s.

This assumption, although not particularly intricate, has one clear departure from the actual decision process we use. In particular, the condition that $\forall \gamma_t > (C\alpha)^{-1}$, $\mathbb{P}(I_t | x_t, \gamma_t) = 0$ is not one we can realistically enforce in practice, because we do not know the value of the smoothness constant $\alpha > 0$. However, this is equivalent to asking for a hard upper bound on the step size γ_t proportional to an inverse Lipschitz constant, which is a very common assumption in SGD analysis [GG23]. In practice, we expect that this departure is not a major issue; the probability of I_t in the case where the objective function is growing is not exactly zero, but should be close enough to zero (and decaying as the objective function increases) so as to not modify the below results meaningfully.

Assumption C.6. Assumption C.4 holds, $\gamma_0 \leq \alpha^{-1}$, and there exist constants $\delta, \ell \in (0, 1)$ such that

$$\frac{C\gamma_t\beta}{2\mu} > f(x_t) \implies \mathbb{P}(R_t | x_t, \gamma_t) \geq 1 - \delta, \quad \frac{C\gamma_t\beta}{2\mu} \leq f(x_t) \implies \mathbb{P}(R_t | x_t, \gamma_t) \leq \delta,$$

and

$$\gamma_t > (C\alpha)^{-1} \implies \mathbb{P}(I_t | x_t, \gamma_t) = 0, \quad \gamma_t \leq (C\alpha)^{-1} \text{ and } \frac{C\gamma_t\beta}{2\ell\mu} < f(x_t) \implies \mathbb{P}(I_t | x_t, \gamma_t) \geq 1 - \delta.$$

Assumption C.7 below has three parts: Eqs. (7) to (9). Eq. (7) requires that we are likely to pick a step size that results in a non-negligible descent, which again is reasonably expected to be satisfied by the decision process in Appendix C.1 in practice. Eq. (8) enforces that when the step size γ_t is small enough (in the “not bad” region mentioned earlier), the decision process takes *at least* an amount of time roughly proportional to $1/f(x_t)$ to make a decision. This should be satisfied for some small enough $u \in (0, 1)$ by most reasonable decision processes that are motivated by streaming statistical hypothesis tests; if $f(x_t)$ is small, the difference between $f(x_t)$ and subsequent descending iterations must also be small (since $f \geq 0$), and so distinguishing which stream is the “winner” should take more samples and hence take longer. Our suggested process in Appendix C.1 should adhere to this behaviour. Eq. (9) simply asserts that when the learning rate is close to the upper threshold of α^{-1} and the function value f is also large, that the decision process guarantees reasonable descent; this again should be satisfied in practice for some large enough w . Note that in the decision process in Appendix C.1, we enforce a minimum number of samples $\tau_t \geq M$, which also incidentally helps enforce Eq. (9). For example, if $M > (\log(c/2))/\log(1 - c\mu/(\alpha C))$, Eq. (9) is automatically satisfied with $w = 1/2$.

Assumption C.7. Assumption C.4 holds and there exist constants $\epsilon, u, w \in (0, 1)$, $u \leq c\mu/C\alpha$ such that if $C\gamma_t\beta/2\mu \leq f(x_t)$,

$$\frac{\gamma_t\beta}{2\mu} \mathbb{E}[\mathbb{1}[I_t]C + \mathbb{1}[S_t] + \mathbb{1}[D_t]c | x_t, \gamma_t] \leq (1 - \delta)\epsilon f(x_t), \quad (7)$$

if $C\gamma_t\beta/2\mu \leq f(x_t)$ and $\gamma_t \leq (C\alpha)^{-1}$,

$$\begin{aligned} & \max\{\mathbb{E}[(1 - C\mu\gamma_t)^{\tau_t} | I_t, x_t, \gamma_t], \mathbb{E}[(1 - \mu\gamma_t)^{\tau_t} | S_t, x_t, \gamma_t], \mathbb{E}[(1 - c\mu\gamma_t)^{\tau_t} | D_t, x_t, \gamma_t]\} \\ & \leq \left(1 - \frac{u\gamma_t C\beta}{2\mu f(x_t)}\right), \end{aligned} \quad (8)$$

and if $C\gamma_t\beta/2\mu \leq f(x_t)$ and $(C\alpha)^{-1} < \gamma_t < \alpha^{-1}$,

$$\max\{\mathbb{E}[(1 - \mu/(\alpha C))^{\tau_t} | S_t, x_t, \gamma_t], \mathbb{E}[(1 - c\mu/(\alpha C))^{\tau_t} | D_t, x_t, \gamma_t]\} \leq wc. \quad (9)$$

The final assumption in Assumption C.8 is the most technical of the assumptions, but is generally quite weak; it simply asserts that it is unlikely that any inner SGD stream terminates at a vanishingly small value of $f(x_t)$ in a single episode. This assumption is likely to hold for almost any reasonable decision process given known results about the geometric ergodicity of the uncorrected Langevin algorithm. Unlike the previous assumptions, this assumption is probably not even necessary, and a more careful analysis might be able to avoid it entirely. In practice, in the off-chance $f(x_t)$ happens to be much smaller than expected, the algorithm will simply try to shrink the step sizes as much as possible until either it successfully returns to the set $a\gamma_t \leq f(x_t)$, or it will accidentally not trigger R_t at some point and immediately $f(x_t)$ will jump larger to a value roughly $\propto \gamma_t$.

Assumption C.8. *There exists $\sigma \geq 0$ such that for all t ,*

$$\sum_{(y,A) \in \{(x_t, \tau_t, D_t), (x_t, \tau_t, S_t), (\bar{x}_t, \tau_t, I_t)\}} \int_0^1 s^{-2} \mathbb{P}(f(y) < saC\gamma_t, A | x_t, \gamma_t) ds \leq \sigma \frac{aC\gamma_t}{f(x_t)} \mathbb{P}(R_t^c | x_t, \gamma_t).$$

We now use these assumptions in Lemmas C.9 to C.11 to verify the conditions of Lemmas C.1 and C.2 and Theorem C.3 and obtain specific values for the constants $a, b, v, \xi, \zeta, \ell, \eta, B$ in terms of the constants $\alpha, \beta, \delta, \epsilon, \mu, u, w, \sigma$ appearing in Assumptions C.4 to C.8. We begin with Lemma C.9, which is a fairly standard result capturing the expected descent for an individual inner stream of SGD iterates.

Lemma C.9. *Suppose Assumption C.4 holds, $0 \leq \gamma \leq \alpha^{-1}$, and $y_0 \in \mathbb{R}^d$. Consider the iteration $y_{k+1} = y_k - \gamma g(y_k, u_k)$, where $u_k \stackrel{iid}{\sim} \phi$. Then for all $j, k \in \mathbb{N}_0$, $j \leq k$, and for all $z \in \mathbb{R}$, $0 < z \leq 2\mu(1 - \frac{1}{2}\gamma\alpha)$,*

$$\mathbb{E}[f(y_k) | y_j] \leq f(y_j)(1 - \gamma z)^{k-j} + (1 - (1 - \gamma z)^{k-j}) \frac{\gamma\beta}{2z}.$$

Lemma C.10 is used to verify the conditions of Lemma C.1, as well as the $f(x_t)(1 - \eta a\gamma_t/f(x_t))$ and $\max\{f(x_t), B\gamma_t\}$ bound conditions in Theorem C.3, with constant values

$$a = \frac{C\beta}{2\mu}, \quad B = \frac{(1 + \delta)C\beta}{2\mu}, \quad b = \frac{\delta C\beta}{2\mu}, \quad \eta = u(1 - \delta)(1 - \epsilon), \quad v = (1 - \delta)c + \delta C. \quad (10)$$

Lemma C.10. *Suppose Assumptions C.4 to C.7 hold. Then if $C\gamma_t\beta/2\mu > f(x_t)$,*

$$\mathbb{E}[f(x_{t+1}) | x_t, \gamma_t] \leq f(x_t) + \gamma_t \frac{\delta C\beta}{2\mu} \quad \text{and} \quad \mathbb{E}[\gamma_{t+1} | x_t, \gamma_t] \leq \gamma_t((1 - \delta)c + \delta C), \quad (11)$$

and otherwise,

$$\mathbb{E}[f(x_{t+1}) | x_t, \gamma_t] \leq \left(1 - \frac{uC\beta\gamma_t}{2\mu f(x_t)}(1 - \delta)(1 - \epsilon)\right) f(x_t). \quad (12)$$

Let $V : \mathbb{R}^{d+1} \rightarrow \mathbb{R}$ be defined as

$$V(x, \gamma) = \frac{\ell f(x)}{a\gamma} \mathbb{1}\left[\frac{\ell f(x)}{a\gamma} > 1\right] + \frac{a\gamma}{f(x)} \mathbb{1}\left[\frac{a\gamma}{f(x)} > 1\right], \quad (13)$$

where ℓ is the constant from Assumption C.6, and $a = \beta C/2\mu$ as per Eq. (10). The final technical lemma is Lemma C.11, which is used to verify the conditions relating to the drift function V in Lemma C.2 and Theorem C.3 with constant values

ℓ = the ℓ from Assumption C.6

$$\xi = \max\left\{\left(w + \frac{(1 - wc)\ell}{C} + \delta\right) \vee \left(\frac{1 - \delta}{C} + \frac{\delta}{c}\right) + \delta c\ell + \sigma C\ell, \ell c^{-1}\delta + c + \sigma\delta C\right\} \quad (14)$$

$$\zeta = c^{-1} + \delta c + \sigma C.$$

Lemma C.11. Suppose Assumptions C.4 to C.8 hold with $\ell \leq c$. Then

$$\mathbb{E}[V(x_{t+1}, \gamma_{t+1}) | x_t, \gamma_t] \leq \xi V(x_t, \gamma_t) + \zeta \mathbf{1}[V(x_t, \gamma_t) \leq 1],$$

where

$$\begin{aligned} \xi &= \max \left\{ \left(w + \frac{(1-wc)\ell}{C} + \delta \right) \vee \left(\frac{1-\delta}{C} + \frac{\delta}{c} \right) + \delta c \ell + \sigma C \ell, \ell c^{-1} \delta + c + \sigma \delta C \right\} \\ \zeta &= c^{-1} + \delta c + \sigma C, \end{aligned}$$

and V is given by Eq. (13).

Theorem C.12 presents the final convergence result, which is just a more specific version of Theorem 4.6 from the main text with all constants and value constraints specified. Note that the theorem only holds when ℓ, δ are small enough, but this is not a major requirement. The constant ℓ is an abstract value not involved in any aspect of the AutoSGD algorithm, and so can be taken as large as possible while still satisfying the bounds in Eq. (15). The constant δ can be tuned to be as small as needed by making the decision process more conservative (e.g., increasing the value of z^* in Algorithm 2).

Theorem C.12. Suppose Assumptions C.4 to C.8 hold with ℓ, δ small enough such that

$$\begin{aligned} \ell &\leq c \\ (1-\delta)c + \delta C &< 1 \\ \max \left\{ \left(w + \frac{(1-wc)\ell}{C} + \delta \right) \vee \left(\frac{1-\delta}{C} + \frac{\delta}{c} \right) + \delta c \ell + \sigma C \ell, \ell c^{-1} \delta + c + \sigma \delta C \right\} &< 1 \\ \text{and } u(1-\delta)(1-\epsilon) - \frac{C\delta}{(1-(1-\delta)c - \delta C)} &> 0. \end{aligned} \quad (15)$$

Let t_n be the n^{th} episode index t at which $\ell f(x_t) \leq \frac{C\beta}{2\mu} \gamma_t \leq f(x_t)$. Then $\forall n \in \mathbb{N}$,

$$\begin{aligned} \mathbb{E}[f(x_{t_n})] &\leq \nu^{n-1} \left(\max \left\{ f(x_0), \frac{C\beta(1+\delta)}{2\mu} \gamma_0 \right\} + \gamma_0 \frac{C^2 \delta \beta}{2\mu(1-(1-\delta)c - \delta C)} \right) \\ \mathbb{E}[t_n] &\leq \frac{\xi V(x_0, \gamma_0) + \zeta}{1-\xi} + (n-1) \frac{\xi + \zeta}{1-\xi}, \end{aligned}$$

where

$$\begin{aligned} \nu &= 1 - \ell \left(u(1-\delta)(1-\epsilon) - \frac{C\delta}{(1-(1-\delta)c - \delta C)} \right) < 1 \\ \xi &= \max \left\{ \left(w + \frac{(1-wc)\ell}{C} + \delta \right) \vee \left(\frac{1-\delta}{C} + \frac{\delta}{c} \right) + \delta c \ell + \sigma C \ell, \ell c^{-1} \delta + c + \sigma \delta C \right\} \\ \zeta &= c^{-1} + \delta c + \sigma C. \end{aligned}$$

Proof. Assumptions C.4 to C.8 with the constant constraints given by Eq. (15) imply that Lemmas C.9 to C.11 hold. These then verify the conditions of Lemmas C.1 and C.2 and Theorem C.3 with the constants specified in Eqs. (10) and (14). (We also use the intermediate bound Eq. (17) from the proof of Lemma C.2.) Substitution yields the stated result. $\square$

Note finally that Theorem C.12 does not say anything about the total number of SGD iterations; in fact, none of the assumptions above preclude the possibility that $\tau_t = \infty$ for some $t \in \mathbb{N}$. That being said, any reasonable decision process should indeed be designed to satisfy $\tau_t < \infty$ a.s. to avoid this pathology. We leave a more detailed analysis of the expected total number of SGD iterations to future work.

C.3 Stochastic convergence proofs

Proof of Lemma C.1. We have that for $j \geq 1$,

$$f(x_{\tau \wedge (t+j)}) = f(x_{t+1}) + \sum_{k=2}^j \mathbf{1}[\tau \geq t+k] (f(x_{t+k}) - f(x_{t+k-1})).$$

Note that since $f \geq 0$, if

$$\mathbb{E} \left[\sum_{k=2}^{\infty} \mathbb{1}[\tau \geq t+k](f(x_{t+k}) + f(x_{t+k-1})) | x_t, \gamma_t \right] < \infty \quad a.s.,$$

then $\sum_{k=2}^{\infty} \mathbb{1}[\tau \geq t+k](f(x_{t+k}) + f(x_{t+k-1})) < \infty$ a.s., which implies the limit $f(x_\tau) = \lim_{k \rightarrow \infty} f(x_{\tau \wedge k})$ is a well-defined random variable, and the dominated convergence theorem guarantees that $\mathbb{E}[f(x_\tau) | x_t, \gamma_t] = \lim_{k \rightarrow \infty} \mathbb{E}[f(x_{t \wedge k}) | x_t, \gamma_t]$ is similarly well-defined.

We now show that the sum is finite. By the definition of τ and Fubini's theorem,

$$\begin{aligned} & \mathbb{E} \left[\sum_{k=2}^{\infty} \mathbb{1}[\tau \geq t+k](f(x_{t+k}) + f(x_{t+k-1})) | x_t, \gamma_t \right] \\ &= \sum_{k=2}^{\infty} \mathbb{E}[\mathbb{1}[\tau \geq t+k](f(x_{t+k}) + f(x_{t+k-1})) | x_t, \gamma_t] \\ &= \sum_{k=2}^{\infty} \mathbb{E}[\mathbb{1}[\tau \geq t+k]f(x_{t+k}) | x_t, \gamma_t] + \mathbb{E}[\mathbb{1}[\tau \geq t+k]f(x_{t+k-1}) | x_t, \gamma_t] \\ &\leq \sum_{k=2}^{\infty} \mathbb{E}[\mathbb{1}[\tau \geq t+k](f(x_{t+k-1}) + b\gamma_{t+k-1}) | x_t, \gamma_t] + \mathbb{E}[\mathbb{1}[\tau \geq t+k]f(x_{t+k-1}) | x_t, \gamma_t]. \end{aligned}$$

Now, since $\tau \geq t+k$, we have $f(x_{t+k-1}) < a\gamma_{t+k-1}$, and so

$$\mathbb{E} \left[\sum_{k=2}^{\infty} \mathbb{1}[\tau \geq t+k](f(x_{t+k}) + f(x_{t+k-1})) | x_t, \gamma_t \right] \leq (2a+b) \sum_{k=2}^{\infty} \mathbb{E}[\mathbb{1}[\tau \geq t+k]\gamma_{t+k-1} | x_t, \gamma_t].$$

The sum on the right hand side can be bounded by noting that $\gamma_{t+1} \leq C\gamma_t$ a.s., extracting the first term, shifting indices, and then bounding the decay in γ_{t+k} using the definition of τ and the law of total expectation:

$$\begin{aligned} \sum_{k=2}^{\infty} \mathbb{E}[\mathbb{1}[\tau \geq t+k]\gamma_{t+k-1} | x_t, \gamma_t] &\leq C\gamma_t \mathbb{P}(\tau > t+1 | x_t, \gamma_t) + \sum_{k=3}^{\infty} \mathbb{E}[\mathbb{1}[\tau \geq t+k]\gamma_{t+k-1} | x_t, \gamma_t] \\ &\leq C\gamma_t \mathbb{P}(\tau > t+1 | x_t, \gamma_t) + \sum_{k=3}^{\infty} \mathbb{E}[\mathbb{1}[\tau \geq t+k-1]\gamma_{t+k-1} | x_t, \gamma_t] \\ &= C\gamma_t \mathbb{P}(\tau > t+1 | x_t, \gamma_t) + \sum_{k=2}^{\infty} \mathbb{E}[\mathbb{1}[\tau \geq t+k]\gamma_{t+k} | x_t, \gamma_t] \\ &\leq C\gamma_t \mathbb{P}(\tau > t+1 | x_t, \gamma_t) + v \sum_{k=2}^{\infty} \mathbb{E}[\mathbb{1}[\tau \geq t+k]\gamma_{t+k-1} | x_t, \gamma_t]. \end{aligned}$$

Noting that the sums on the left and right hand sides are the same, we have that

$$\mathbb{E} \left[\sum_{k=2}^{\infty} \mathbb{1}[\tau \geq t+k](f(x_{t+k}) + f(x_{t+k-1})) | x_t, \gamma_t \right] \leq \frac{(2a+b)C\gamma_t \mathbb{P}(\tau > t+1)}{1-v} < \infty \quad a.s.$$

Therefore, $f(x_\tau)$ and $\mathbb{E}[f(x_\tau) | x_t, \gamma_t]$ are well-defined (as stated earlier) and

$$\mathbb{E}[f(x_\tau) | x_t, \gamma_t] = \mathbb{E}[f(x_{t+1}) | x_t, \gamma_t] + \sum_{k=2}^{\infty} \mathbb{E}[\mathbb{1}[\tau \geq t+k](f(x_{t+k}) - f(x_{t+k-1})) | x_t, \gamma_t].$$

To obtain the bound in the result, we now apply the same technique as in the earlier analysis, except that we maintain the negative sign on the second f term in the sum:

$$\begin{aligned} & \mathbb{E}[f(x_\tau) | x_t, \gamma_t] \\ &\leq \mathbb{E}[f(x_{t+1}) | x_t, \gamma_t] + \sum_{k=2}^{\infty} \mathbb{E}[\mathbb{1}[\tau \geq t+k](f(x_{t+k-1}) + b\gamma_{t+k-1} - f(x_{t+k-1})) | x_t, \gamma_t] \end{aligned}$$

$$\begin{aligned}
&= \mathbb{E}[f(x_{t+1})|x_t, \gamma_t] + b \sum_{k=2}^{\infty} \mathbb{E}[\mathbb{1}[\tau \geq t+k] \gamma_{t+k-1} | x_t, \gamma_t] \\
&\leq \mathbb{E}[f(x_{t+1})|x_t, \gamma_t] + b \left(C \gamma_t \mathbb{P}(\tau > t+1 | x_t, \gamma_t) + \sum_{k=3}^{\infty} \mathbb{E}[\mathbb{1}[\tau \geq t+k] \gamma_{t+k-1} | x_t, \gamma_t] \right) \\
&\leq \mathbb{E}[f(x_{t+1})|x_t, \gamma_t] + \frac{bC \gamma_t \mathbb{P}(\tau > t+1 | x_t, \gamma_t)}{1-v} \\
&\leq \mathbb{E}[f(x_{t+1})|x_t, \gamma_t] + \frac{bC \gamma_t}{1-v}.
\end{aligned}$$

□

Proof of Lemma C.2. For notational brevity in the proof, let $z_t = (x_t, \gamma_t)$ and $V_t = V(z_t) = V(x_t, \gamma_t)$. By definition of τ ,

$$\begin{aligned}
\mathbb{P}(\tau > k | z_t) &= \mathbb{E} \left[\mathbb{1} \left[\min_{j \in \{1, \dots, k\}} V_{t+j} > 1 \right] | z_t \right] \\
&= \mathbb{E} \left[\mathbb{E}[\mathbb{1}[V_{t+k} > 1] | z_{t+k-1}] \mathbb{1} \left[\min_{j \in \{1, \dots, k-1\}} V_{t+j} > 1 \right] | z_t \right] \\
&\leq \mathbb{E} \left[\mathbb{E}[V_{t+k} | z_{t+k-1}] \mathbb{1} \left[\min_{j \in \{1, \dots, k-1\}} V_{t+j} > 1 \right] | z_t \right] \\
&\leq \xi \mathbb{E} \left[V_{t+k-1} \mathbb{1} \left[\min_{j \in \{1, \dots, k-1\}} V_{t+j} > 1 \right] | z_t \right] \\
&\leq \xi \mathbb{E} \left[V_{t+k-1} \mathbb{1} \left[\min_{j \in \{1, \dots, k-2\}} V_{t+j} > 1 \right] | z_t \right].
\end{aligned}$$

Continuing this logic yields

$$\mathbb{P}(\tau > k | z_t, \gamma_t) \leq \xi^{k-1} \mathbb{E}[V_{t+1} | z_t] \leq \xi^{k-1} (\xi V_t + \zeta \mathbb{1}[V_t \leq 1]).$$

□

Proof of Theorem C.3. Let $t'_i, i \in \mathbb{N}$ be the i^{th} index $t \in \mathbb{N}_0$ at which $a\gamma_t \leq f(x_t)$ (or ∞ if no such index exists); Lemma C.1 and Eq. (6) imply that

$$\begin{aligned}
\mathbb{E}[f(x_{t'_1}) | x_0, \gamma_0] &\leq \max\{f(x_0), B\gamma_0\} + \gamma_0 \frac{Cb}{1-v} \\
\forall i \in \mathbb{N}, \quad \mathbb{E}[f(x_{t'_{i+1}}) | x_{t'_i}, \gamma_{t'_i}] &\leq f(x_{t'_i}) \left(1 - \frac{a\gamma_{t'_i}}{f(x_{t'_i})} \left(\eta - \frac{Cb}{a(1-v)} \right) \right).
\end{aligned}$$

By Eq. (6), the multiplicative factor is at most 1; hence $f(x_{t'_i})$ is a nonnegative supermartingale. Let $i_n, n \in \mathbb{N}$ be the n^{th} index i at which $V(x_{t'_i}, \gamma_{t'_i}) \leq 1$. Then by the optional stopping theorem for nonnegative supermartingales [Dur10, Theorem 5.7.6] and Eq. (6),

$$\begin{aligned}
\mathbb{E}[f(x_{t'_{i_n+1}}) | x_{t'_{i_n}}, \gamma_{t'_{i_n}}] &= \mathbb{E} \left[\mathbb{E}[f(x_{t'_{i_n+1}}) | x_{t'_{i_n+1}}, \gamma_{t'_{i_n+1}}] | x_{t'_{i_n}}, \gamma_{t'_{i_n}} \right] \\
&\leq \mathbb{E} \left[f(x_{t'_{i_n+1}}) | x_{t'_{i_n}}, \gamma_{t'_{i_n}} \right] \\
&\leq f(x_{t'_{i_n}}) \left(1 - \frac{a\gamma_{t'_{i_n}}}{f(x_{t'_{i_n}})} \left(\eta - \frac{Cb}{a(1-v)} \right) \right) \\
&\leq f(x_{t'_{i_n}}) \left(1 - \ell \left(\eta - \frac{Cb}{a(1-v)} \right) \right).
\end{aligned}$$

Therefore,

$$\mathbb{E}[f(x_{t'_{i_n}})] \leq \nu^{n-1} \mathbb{E}[f(x_{t'_1})] \leq \nu^{n-1} \mathbb{E}[f(x_{t'_1})] \leq \nu^{n-1} f_0.$$

The first result follows by setting $t_n = t'_{i_n}$. Next, by Lemma C.2, we have that

$$\begin{aligned}
\mathbb{E}[t_n|x_0, \gamma_0] &= \sum_{k=1}^n \mathbb{E}[t_k - t_{k-1}|x_0, \gamma_0] \\
&= \mathbb{E}[t_1 - t_0|x_0, \gamma_0] + \sum_{k=2}^n \mathbb{E}[\mathbb{E}[t_k - t_{k-1}|x_{t_{k-1}}, \gamma_{t_{k-1}}]|x_0, \gamma_0] \\
&\leq \frac{\xi V(x_0, \gamma_0) + \zeta \mathbf{1}[V(x_0, \gamma_0) \leq 1]}{1 - \xi} + \sum_{k=2}^n \mathbb{E}\left[\frac{\xi + \zeta}{1 - \xi}|x_0, \gamma_0\right] \\
&= \frac{\xi V(x_0, \gamma_0) + \zeta \mathbf{1}[V(x_0, \gamma_0) \leq 1]}{1 - \xi} + (n-1) \frac{\xi + \zeta}{1 - \xi} \\
&= \frac{V_0}{1 - \xi} + (n-1) \frac{\xi + \zeta}{1 - \xi}.
\end{aligned}$$

□

Proof of Lemma C.9. Since f is twice continuously differentiable by Assumption C.4, the Taylor remainder theorem [Apo67, Theorem 7.6] and iteration $y_{k+1} = y_k - \gamma g(y_k, u_k)$ yield

$$\begin{aligned}
f(y_{k+1}) &= f(y_k) + \nabla f(y_k)^T (y_{k+1} - y_k) \\
&\quad + (y_{k+1} - y_k)^T \int_0^1 (1-u) \nabla^2 f((1-u)y_k + uy_{k+1}) du (y_{k+1} - y_k) \\
&= f(y_k) - \gamma \nabla f(y_k)^T g(y_k, u_k) + \frac{1}{2} \gamma^2 g(y_k, u_k)^T \int_0^1 2(1-s) \nabla^2 f(y_k - s\gamma g_k) ds g(y_k, u_k),
\end{aligned}$$

where $g_k := g(y_k, u_k)$. Then by the unbiasedness of $g(y_k, u_k)$ for $\nabla f(y_k)$ and the moment bound in Assumption C.4,

$$\begin{aligned}
\mathbb{E}[f(y_{k+1})|y_k] &\leq f(y_k) - \gamma \|\nabla f(y_k)\|^2 + \frac{1}{2} \gamma^2 (\alpha \|\nabla f(y_k)\|^2 + \beta) \\
&= f(y_k) - \left(\gamma - \frac{1}{2} \gamma^2 \alpha\right) \|\nabla f(y_k)\|^2 + \frac{1}{2} \gamma^2 \beta \\
&\leq f(y_k) \left(1 - 2\mu\gamma \left(1 - \frac{1}{2} \gamma \alpha\right)\right) + \frac{1}{2} \gamma^2 \beta.
\end{aligned}$$

Since $f(y_k) \geq 0$, for all $0 < z \leq 2\mu(1 - \frac{1}{2}\gamma\alpha)$,

$$\mathbb{E}[f(y_{k+1})|y_k] \leq f(y_k)(1 - \gamma z) + \frac{1}{2} \gamma^2 \beta.$$

Recurring this bound yields, for $j \leq k$,

$$\begin{aligned}
\mathbb{E}[f(y_k)|y_j] &\leq f(y_j)(1 - \gamma z)^{k-j} + \sum_{i=0}^{k-j-1} (1 - \gamma z)^i \frac{1}{2} \gamma^2 \beta \\
&= f(y_j)(1 - \gamma z)^{k-j} + (1 - (1 - \gamma z)^{k-j}) \frac{\gamma \beta}{2z}.
\end{aligned}$$

□

Proof of Lemma C.10. By the definition of γ_{t+1} ,

$$\begin{aligned}
\mathbb{E}[\gamma_{t+1}|x_t, \gamma_t] &= \mathbb{E}[\gamma_t(I_t C + S_t + (D_t + R_t)c)|x_t, \gamma_t] \\
&= \gamma_t(C\mathbb{P}(I_t|x_t, \gamma_t) + \mathbb{P}(S_t|x_t, \gamma_t) + c\mathbb{P}(D_t \cup R_t|x_t, \gamma_t)).
\end{aligned}$$

If $C\gamma_t\beta/2\mu > f(x_t)$, we have $\mathbb{P}(R_t|x_t, \gamma_t) \geq 1 - \delta$ by Assumption C.6, and hence

$$\mathbb{E}[\gamma_{t+1}|x_t, \gamma_t] \leq \gamma_t(\max\{C, 1\}\mathbb{P}(I_t \cup S_t|x_t, \gamma_t) + c\mathbb{P}(D_t \cup R_t|x_t, \gamma_t))$$

$$\leq \gamma_t(C\delta + c(1 - \delta)).$$

By definition of $f(x_{t+1})$,

$$\begin{aligned} & \mathbb{E}[f(x_{t+1})|x_t, \gamma_t] \\ &= \mathbb{E}[\mathbb{1}[I_t]f(\bar{x}_{t,\tau_t}) + \mathbb{1}[S_t]f(x_{t,\tau_t}) + \mathbb{1}[D_t]f(\underline{x}_{t,\tau_t}) + \mathbb{1}[R_t]f(x_t)|x_t, \gamma_t] \\ &= \mathbb{E}\left[\sum_{k=1}^{\infty} \{\mathbb{1}[I_t, \tau_t = k]f(\bar{x}_{t,k}) + \mathbb{1}[S_t, \tau_t = k]f(x_{t,k}) + \mathbb{1}[D_t, \tau_t = k]f(\underline{x}_{t,k})\} + \mathbb{1}[R_t]f(x_t)|x_t, \gamma_t\right]. \end{aligned}$$

As the summands are nonnegative, we can interchange the expectation and sum, and by Assumption C.5,

$$\begin{aligned} & \mathbb{E}[f(x_{t+1})|x_t, \gamma_t] \\ & \leq \sum_{k=1}^{\infty} \{\mathbb{P}(I_t, \tau_t = k|x_t, \gamma_t)\mathbb{E}[f(\bar{x}_{t,k})|x_t, \gamma_t] + \mathbb{P}(S_t, \tau_t = k|x_t, \gamma_t)\mathbb{E}[f(x_{t,k})|x_t, \gamma_t] \\ & \quad + \mathbb{P}(D_t, \tau_t = k|x_t, \gamma_t)\mathbb{E}[f(\underline{x}_{t,k})|x_t, \gamma_t]\} + \mathbb{P}(R_t|x_t, \gamma_t)f(x_t). \end{aligned}$$

We have that $\gamma_t \leq \alpha^{-1}$ a.s. by Assumption C.6. If $\gamma_t \leq (C\alpha)^{-1}$, then

$$\min_{z \in \{c, 1, C\}} 2\mu \left(1 - z \frac{1}{2} \gamma_t \alpha\right) \geq 2\mu \left(1 - \frac{1}{2} C \gamma_t \alpha\right) \geq \mu,$$

and the same result holds for $\min_{z \in \{c, 1\}} \gamma_t \geq (C\alpha)^{-1}$. So applying Lemma C.9 with γ set to $c\gamma_t$, γ_t , and $C\gamma_t$ yields the three bounds

$$\begin{aligned} \mathbb{E}[f(x_{t,k})|x_t, \gamma_t] &\leq f(x_t)(1 - \mu\gamma_t)^k + (1 - (1 - \mu\gamma_t)^k) \frac{\gamma_t \beta}{2\mu} \\ \mathbb{E}[f(\underline{x}_{t,k})|x_t, \gamma_t] &\leq f(x_t)(1 - c\mu\gamma_t)^k + (1 - (1 - c\mu\gamma_t)^k) \frac{c\gamma_t \beta}{2\mu} \\ \text{if } \gamma_t \leq (C\alpha)^{-1}, \quad \mathbb{E}[f(\bar{x}_{t,k})|x_t, \gamma_t] &\leq f(x_t)(1 - C\mu\gamma_t)^k + (1 - (1 - C\mu\gamma_t)^k) \frac{C\gamma_t \beta}{2\mu}. \end{aligned}$$

Therefore,

$$\begin{aligned} & \mathbb{E}[f(x_{t+1})|x_t, \gamma_t] \\ & \leq \mathbb{P}(R_t|x_t, \gamma_t)f(x_t) \\ & \quad + \sum_{k=1}^{\infty} \mathbb{P}(I_t, \tau_t = k|x_t, \gamma_t) \left(f(x_t)(1 - C\mu\gamma_t)^k + (1 - (1 - C\mu\gamma_t)^k) \frac{C\gamma_t \beta}{2\mu} \right) \\ & \quad + \sum_{k=1}^{\infty} \mathbb{P}(S_t, \tau_t = k|x_t, \gamma_t) \left(f(x_t)(1 - \mu\gamma_t)^k + (1 - (1 - \mu\gamma_t)^k) \frac{\gamma_t \beta}{2\mu} \right) \\ & \quad + \sum_{k=1}^{\infty} \mathbb{P}(D_t, \tau_t = k|x_t, \gamma_t) \left(f(x_t)(1 - c\mu\gamma_t)^k + (1 - (1 - c\mu\gamma_t)^k) \frac{c\gamma_t \beta}{2\mu} \right), \quad (16) \end{aligned}$$

where the I_t term holds since $\mathbb{P}(I_t|x_t, \gamma_t) = 0$ when $\gamma_t \geq (C\alpha)^{-1}$ by Assumption C.6. First consider the case where $C\gamma_t \beta / (2\mu) > f(x_t)$. In this case, we have $\mathbb{P}(R_t|x_t, \gamma_t) \geq 1 - \delta$ by Assumption C.6, and so Eq. (16) yields

$$\begin{aligned} & \mathbb{E}[f(x_{t+1})|x_t, \gamma_t] \\ & \leq \mathbb{P}(R_t|x_t, \gamma_t)f(x_t) + \sum_{k=1}^{\infty} \mathbb{P}(I_t, \tau_t = k|x_t, \gamma_t) \max \left\{ f(x_t), \frac{C\gamma_t \beta}{2\mu} \right\} \\ & \quad + \sum_{k=1}^{\infty} \mathbb{P}(S_t, \tau_t = k|x_t, \gamma_t) \max \left\{ f(x_t), \frac{\gamma_t \beta}{2\mu} \right\} + \sum_{k=1}^{\infty} \mathbb{P}(D_t, \tau_t = k|x_t, \gamma_t) \max \left\{ f(x_t), \frac{c\gamma_t \beta}{2\mu} \right\} \\ & \leq \mathbb{P}(R_t|x_t, \gamma_t)f(x_t) + \mathbb{P}(R_t^c|x_t, \gamma_t) \max \left\{ f(x_t), \frac{C\gamma_t \beta}{2\mu}, \frac{\gamma_t \beta}{2\mu}, \frac{c\gamma_t \beta}{2\mu} \right\} \end{aligned}$$

$$\begin{aligned}
&\leq \mathbb{P}(R_t|x_t, \gamma_t)f(x_t) + \mathbb{P}(R_t^c|x_t, \gamma_t)\frac{C\gamma_t\beta}{2\mu} \\
&\leq f(x_t) + \gamma_t\delta\frac{C\beta}{2\mu}.
\end{aligned} \tag{17}$$

This completes the results stated in Eq. (11). Finally, consider the case where $C\gamma_t\beta/2\mu \leq f(x_t)$. We split this into two subcases: one where $\gamma_t \leq (C\alpha)^{-1}$, and one where $(C\alpha)^{-1} < \gamma_t \leq \alpha^{-1}$. In the subcase where $\gamma_t \leq (C\alpha)^{-1}$, by Eq. (16) and Assumptions C.6 and C.7,

$$\begin{aligned}
&\mathbb{E}[f(x_{t+1})|x_t, \gamma_t] \\
&\leq \mathbb{P}(R_t|x_t, \gamma_t)f(x_t) + \left(1 - \frac{uC\beta\gamma_t}{2\mu f(x_t)}\right)\mathbb{P}(R_t^c|x_t, \gamma_t)f(x_t) \\
&\quad + \frac{uC\beta\gamma_t}{2\mu f(x_t)}\frac{\gamma_t\beta}{2\mu}\mathbb{E}[\mathbb{1}[I_t]C + \mathbb{1}[S_t] + \mathbb{1}[D_t]c|x_t, \gamma_t] \\
&\leq \delta f(x_t) + \left(1 - \frac{uC\beta\gamma_t}{2\mu f(x_t)}\right)(1 - \delta)f(x_t) + \frac{uC\beta\gamma_t}{2\mu f(x_t)}\frac{\gamma_t\beta}{2\mu}\mathbb{E}[\mathbb{1}[I_t]C + \mathbb{1}[S_t] + \mathbb{1}[D_t]c|x_t, \gamma_t] \\
&\leq \delta f(x_t) + \left(1 - \frac{uC\beta\gamma_t}{2\mu f(x_t)}\right)(1 - \delta)f(x_t) + \frac{uC\beta\gamma_t}{2\mu f(x_t)}(1 - \delta)\epsilon f(x_t) \\
&= \left(1 - \frac{uC\beta\gamma_t}{2\mu f(x_t)}(1 - \delta)(1 - \epsilon)\right)f(x_t).
\end{aligned}$$

In the subcase where $(C\alpha)^{-1} < \gamma_t \leq \alpha^{-1}$, since $\tau_t \geq 1$,

$$\max\{\mathbb{E}[(1 - \mu\gamma_t)^{\tau_t}|S_t, x_t, \gamma_t], \mathbb{E}[(1 - c\mu\gamma_t)^{\tau_t}|D_t, x_t, \gamma_t]\} \leq \left(1 - \frac{c\mu}{C\alpha}\right) \leq (1 - u),$$

and so by Eq. (16) and Assumptions C.6 and C.7,

$$\begin{aligned}
&\mathbb{E}[f(x_{t+1})|x_t, \gamma_t] \\
&\leq \mathbb{P}(R_t|x_t, \gamma_t)f(x_t) + (1 - u)\mathbb{P}(R_t^c|x_t, \gamma_t)f(x_t) + u\frac{\gamma_t\beta}{2\mu}\mathbb{E}[\mathbb{1}[I_t]C + \mathbb{1}[S_t] + \mathbb{1}[D_t]c|x_t, \gamma_t] \\
&\leq \delta f(x_t) + (1 - u)(1 - \delta)f(x_t) + u(1 - \delta)\epsilon f(x_t) \\
&= (1 - u(1 - \delta)(1 - \epsilon))f(x_t) \\
&\leq \left(1 - \frac{uC\gamma_t\beta}{2\mu f(x_t)}(1 - \delta)(1 - \epsilon)\right)f(x_t).
\end{aligned}$$

This completes the result stated in Eq. (12). $\square$

Proof of Lemma C.II. For all $t \in \mathbb{N}_0$, denote $V_t = V(x_t, \gamma_t)$ for brevity.

$$\mathbb{E}[V_{t+1}|x_t, \gamma_t] = \mathbb{E}\left[\frac{\ell f(x_{t+1})}{a\gamma_{t+1}}\mathbb{1}\left[\frac{\ell f(x_{t+1})}{a\gamma_{t+1}} > 1\right] + \frac{a\gamma_{t+1}}{f(x_{t+1})}\mathbb{1}\left[\frac{a\gamma_{t+1}}{f(x_{t+1})} > 1\right]|x_t, \gamma_t\right].$$

We begin by analyzing the first term. By definition of $f(x_{t+1})$,

$$\begin{aligned}
&\mathbb{E}\left[\frac{\ell f(x_{t+1})}{a\gamma_{t+1}}\mathbb{1}\left[\frac{\ell f(x_{t+1})}{a\gamma_{t+1}} > 1\right]|x_t, \gamma_t\right] \\
&= \mathbb{E}\left[\frac{\ell(I_t + S_t + D_t + R_t)f(x_{t+1})}{a\gamma_{t+1}}\mathbb{1}\left[\frac{\ell f(x_{t+1})}{a\gamma_{t+1}} > 1\right]|x_t, \gamma_t\right] \\
&\leq \mathbb{E}\left[\frac{\ell\left(I_t + S_t + D_t + R_t\mathbb{1}\left[\frac{\ell f(x_{t+1})}{a\gamma_{t+1}} > 1\right]\right)f(x_{t+1})}{a\gamma_{t+1}}|x_t, \gamma_t\right] \\
&= \mathbb{E}\left[\frac{I_t\ell f(\bar{x}_t, \tau_t)}{aC\gamma_t}|x_t, \gamma_t\right] + \mathbb{E}\left[\frac{S_t\ell f(x_t, \tau_t)}{a\gamma_t}|x_t, \gamma_t\right] + \mathbb{E}\left[\frac{D_t\ell f(x_t, \tau_t)}{ac\gamma_t}|x_t, \gamma_t\right] \\
&\quad + \mathbb{E}\left[\frac{R_t\ell f(x_t)}{ac\gamma_t}\mathbb{1}\left[\frac{\ell f(x_{t+1})}{a\gamma_{t+1}} > 1\right]|x_t, \gamma_t\right].
\end{aligned}$$

Then by Assumption C.5 and Lemma C.9,

$$\begin{aligned}
&\leq \mathbb{P}\left(R_t, \frac{\ell f(x_{t+1})}{a\gamma_{t+1}} > 1 | x_t, \gamma_t\right) \frac{\ell f(x_t)}{ac\gamma_t} \\
&+ \sum_{k=1}^{\infty} \frac{\ell \mathbb{P}(I_t, \tau_t = k | x_t, \gamma_t)}{aC\gamma_t} (f(x_t)(1 - C\mu\gamma_t)^k + (1 - (1 - C\mu\gamma_t)^k)a\gamma_t) \\
&+ \sum_{k=1}^{\infty} \frac{\ell \mathbb{P}(S_t, \tau_t = k | x_t, \gamma_t)}{a\gamma_t} \left(f(x_t)(1 - \mu\gamma_t)^k + (1 - (1 - \mu\gamma_t)^k)\frac{a}{C}\gamma_t\right) \\
&+ \sum_{k=1}^{\infty} \frac{\ell \mathbb{P}(D_t, \tau_t = k | x_t, \gamma_t)}{ac\gamma_t} \left(f(x_t)(1 - c\mu\gamma_t)^k + (1 - (1 - c\mu\gamma_t)^k)\frac{ca}{C}\gamma_t\right),
\end{aligned}$$

where the I_t term holds since $\mathbb{P}(I_t | x_t, \gamma_t) = 0$ when $\gamma_t \geq (C\alpha)^{-1}$ by Assumption C.6. When $a\gamma_t < \ell f(x_t)$ and $\gamma_t \leq (C\alpha)^{-1}$, we have $\mathbb{P}(I_t | x_t, \gamma_t) \geq 1 - \delta$ by Assumption C.6. Therefore,

$$\begin{aligned}
&\mathbb{E}\left[\frac{\ell f(x_{t+1})}{a\gamma_{t+1}} \mathbb{1}\left[\frac{\ell f(x_{t+1})}{a\gamma_{t+1}} > 1\right] | x_t, \gamma_t\right] \\
&\leq \frac{\ell f(x_t)}{a\gamma_t} (C^{-1}\mathbb{P}(I_t | x_t, \gamma_t) + \mathbb{P}(S_t | x_t, \gamma_t) + c^{-1}\mathbb{P}(D_t | x_t, \gamma_t) + \mathbb{P}(R_t | x_t, \gamma_t)) \\
&\leq \frac{\ell f(x_t)}{a\gamma_t} \left(\frac{1 - \delta}{C} + \frac{\delta}{c}\right). \tag{18}
\end{aligned}$$

On the other hand, when $a\gamma_t < \ell f(x_t)$ and $(C\alpha)^{-1} < \gamma_t \leq \alpha^{-1}$, we have $\mathbb{P}(I_t | x_t, \gamma_t) = 0$ and $\mathbb{P}(R_t | x_t, \gamma_t) \leq \delta$ by Assumption C.6 and

$$\max\{\mathbb{E}[(1 - \mu\gamma_t)^{\tau_t} | S_t, x_t, \gamma_t], \mathbb{E}[(1 - c\mu\gamma_t)^{\tau_t} | D_t, x_t, \gamma_t]\} \leq wc$$

by Assumption C.7. Therefore,

$$\begin{aligned}
&\mathbb{E}\left[\frac{\ell f(x_{t+1})}{a\gamma_{t+1}} \mathbb{1}\left[\frac{\ell f(x_{t+1})}{a\gamma_{t+1}} > 1\right] | x_t, \gamma_t\right] \\
&\leq \frac{\ell f(x_t)}{a\gamma_t} \left((wc + (1 - wc)\frac{\ell}{C})\mathbb{P}(S_t | x_t, \gamma_t) + c^{-1}(wc + (1 - wc)\frac{c\ell}{C})\mathbb{P}(D_t | x_t, \gamma_t) + \delta\right) \\
&\leq \frac{\ell f(x_t)}{a\gamma_t} \left(w + \frac{(1 - wc)\ell}{C} + \delta\right). \tag{19}
\end{aligned}$$

When $a\gamma_t > f(x_t)$, we have $\mathbb{P}(R_t | x_t, \gamma_t) \geq 1 - \delta$ by Assumption C.6. Furthermore, if $R_t = 1$, then $\frac{\ell f(x_{t+1})}{a\gamma_{t+1}} = \frac{\ell f(x_t)}{ac\gamma_t}$; therefore, as long as $\ell \leq c$, $R_t = 1$ and $\frac{\ell f(x_{t+1})}{a\gamma_{t+1}} > 1$ cannot simultaneously occur when $a\gamma_t > f(x_t)$. So when $a\gamma_t > f(x_t)$,

$$\mathbb{E}\left[\frac{\ell f(x_{t+1})}{a\gamma_{t+1}} \mathbb{1}\left[\frac{\ell f(x_{t+1})}{a\gamma_{t+1}} > 1\right] | x_t, \gamma_t\right] \leq a\gamma_t \frac{\ell}{a\gamma_t} c^{-1} \delta = \ell c^{-1} \delta \leq \ell c^{-1} \delta \frac{a\gamma_t}{f(x_t)}. \tag{20}$$

Finally, when $\ell f(x_t) \leq a\gamma_t \leq f(x_t)$,

$$\mathbb{E}\left[\frac{\ell f(x_{t+1})}{a\gamma_{t+1}} \mathbb{1}\left[\frac{\ell f(x_{t+1})}{a\gamma_{t+1}} > 1\right] | x_t, \gamma_t\right] \leq \frac{\ell f(x_t)}{a\gamma_t} c^{-1} \leq c^{-1}. \tag{21}$$

Next, we analyze the second term. Begin by noting that

$$\begin{aligned}
&\mathbb{E}\left[\frac{a\gamma_{t+1}}{f(x_{t+1})} \mathbb{1}\left[\frac{a\gamma_{t+1}}{f(x_{t+1})} > 1\right] | x_t, \gamma_t\right] \\
&= \int_0^{\infty} \mathbb{P}\left(\frac{a\gamma_{t+1}}{f(x_{t+1})} \mathbb{1}\left[\frac{a\gamma_{t+1}}{f(x_{t+1})} > 1\right] > u | x_t, \gamma_t\right) du \\
&= \int_1^{\infty} \mathbb{P}\left(\frac{a\gamma_{t+1}}{f(x_{t+1})} > u | x_t, \gamma_t\right) du
\end{aligned}$$

$$\begin{aligned}
&= \int_1^\infty \left(\mathbb{P}\left(\frac{ac\gamma_t}{f(x_t)} > u, R_t|x_t, \gamma_t\right) + \mathbb{P}\left(\frac{a\gamma_{t+1}}{f(x_{t+1})} > u, R_t^c|x_t, \gamma_t\right) \right) du \\
&= \mathbb{P}(R_t|x_t, \gamma_t) \left(1 \vee \frac{ac\gamma_t}{f(x_t)} - 1\right) + \int_1^\infty \mathbb{P}\left(\frac{a\gamma_{t+1}}{f(x_{t+1})} > u, R_t^c|x_t, \gamma_t\right) du \\
&= \mathbb{P}(R_t|x_t, \gamma_t) \left(1 \vee \frac{ac\gamma_t}{f(x_t)} - 1\right) + \int_0^1 s^{-2} \mathbb{P}(f(x_{t+1}) < sa\gamma_{t+1}, R_t^c|x_t, \gamma_t) ds \quad (u = s^{-1}) \\
&= \mathbb{P}(R_t|x_t, \gamma_t) \left(1 \vee \frac{ac\gamma_t}{f(x_t)} - 1\right) + \int_0^1 s^{-2} \mathbb{P}(f(\bar{x}_{t,\tau_t}) < saC\gamma_t, I_t|x_t, \gamma_t) ds \\
&\quad + \int_0^1 s^{-2} \mathbb{P}(f(x_{t,\tau_t}) < sa\gamma_t, S_t|x_t, \gamma_t) ds + \int_0^1 s^{-2} \mathbb{P}(f(\underline{x}_{t,\tau_t}) < sac\gamma_t, D_t|x_t, \gamma_t) ds.
\end{aligned}$$

By Assumption C.8,

$$\mathbb{E}\left[\frac{a\gamma_{t+1}}{f(x_{t+1})} \mathbb{1}\left[\frac{a\gamma_{t+1}}{f(x_{t+1})} > 1\right] | x_t, \gamma_t\right] \leq \mathbb{P}(R_t|x_t, \gamma_t) \left(1 \vee \frac{ac\gamma_t}{f(x_t)} - 1\right) + \sigma \frac{aC\gamma_t}{f(x_t)} \mathbb{P}(R_t^c|x_t, \gamma_t).$$

When $a\gamma_t < \ell f(x_t)$, by Assumption C.6 we have that $\mathbb{P}(R_t|x_t, \gamma_t) < \delta$, so

$$\begin{aligned}
\mathbb{E}\left[\frac{a\gamma_{t+1}}{f(x_{t+1})} \mathbb{1}\left[\frac{a\gamma_{t+1}}{f(x_{t+1})} > 1\right] | x_t, \gamma_t\right] &\leq \delta \frac{ac\gamma_t}{f(x_t)} + \sigma \frac{aC\gamma_t}{f(x_t)} \\
&\leq \delta c\ell + \sigma C\ell \\
&\leq (\delta c\ell + \sigma C\ell) \frac{\ell f(x_t)}{a\gamma_t}. \tag{22}
\end{aligned}$$

When $a\gamma_t > f(x_t)$, we have that $\mathbb{P}(R_t^c|x_t, \gamma_t) < \delta$ by Assumption C.6, so

$$\begin{aligned}
\mathbb{E}\left[\frac{a\gamma_{t+1}}{f(x_{t+1})} \mathbb{1}\left[\frac{a\gamma_{t+1}}{f(x_{t+1})} > 1\right] | x_t, \gamma_t\right] &\leq \frac{ac\gamma_t}{f(x_t)} + \sigma \delta C \frac{a\gamma_t}{f(x_t)} \\
&= (c + \sigma \delta C) \frac{a\gamma_t}{f(x_t)}. \tag{23}
\end{aligned}$$

Finally, when $\ell f(x_t) \leq a\gamma_t \leq f(x_t)$, again we have that $\mathbb{P}(R_t|x_t, \gamma_t) < \delta$ by Assumption C.6, so

$$\mathbb{E}\left[\frac{a\gamma_{t+1}}{f(x_{t+1})} \mathbb{1}\left[\frac{a\gamma_{t+1}}{f(x_{t+1})} > 1\right] | x_t, \gamma_t\right] \leq \delta \frac{ac\gamma_t}{f(x_t)} + \sigma \frac{aC\gamma_t}{f(x_t)} \leq \delta c + \sigma C. \tag{24}$$

When $a\gamma_t < \ell f(x_t)$, combining Eqs. (18), (19) and (22) yields

$$\begin{aligned}
\mathbb{E}[V_{t+1}|x_t, \gamma_t] &\leq \left(\left(w + \frac{(1-wc)\ell}{C} + \delta \right) \vee \left(\frac{1-\delta}{C} + \frac{\delta}{c} \right) \right) \frac{\ell f(x_t)}{a\gamma_t} + (\delta c\ell + \sigma C\ell) \frac{\ell f(x_t)}{a\gamma_t} \\
&= \left(\left(w + \frac{(1-wc)\ell}{C} + \delta \right) \vee \left(\frac{1-\delta}{C} + \frac{\delta}{c} \right) + \delta c\ell + \sigma C\ell \right) V_t.
\end{aligned}$$

When $a\gamma_t > f(x_t)$, combining Eqs. (20) and (23) yields

$$\begin{aligned}
\mathbb{E}[V_{t+1}|x_t, \gamma_t] &\leq \ell c^{-1} \delta \frac{a\gamma_t}{f(x_t)} + (c + \sigma \delta C) \frac{a\gamma_t}{f(x_t)} \\
&= (\ell c^{-1} \delta + c + \sigma \delta C) V_t.
\end{aligned}$$

When $\ell f(x_t) \leq a\gamma_t \leq f(x_t)$, combining Eqs. (21) and (24) yields

$$\mathbb{E}[V_{t+1}|x_t, \gamma_t] \leq c^{-1} + \delta c + \sigma C.$$

□

D Constant memory round-based averaging

In this section we present a round-based iterate averaging scheme that has two desirable properties. First, it approximately preserves the final iterate convergence rates Theorem D.1. This is in contrast to standard averaging schemes such as full-trace Polyak or polynomial averaging [SZ13], which can guarantee only $O(1/t)$ convergence due to the bias of the initial terms. Second, the iterate average can be computed efficiently in an online fashion with constant memory in the length of the trace and can be thought of as a constant-memory implementation of an average of the last $\delta \times 100\%$ of iterates. This approach to averaging is known as suffix averaging or tail averaging in the literature [JKK⁺18, RSS11, MNR19].

Our contribution is most similar to that of [Rou19], with slightly different weighting schemes and decision times for when to start a new round of averages. The key idea of our method is to compute two streams of online averages of iterates and then interpolate between them to mimic the desired behaviour. We use a round-based procedure to discard past iterates to ensure the initial bias of early iterates is dropped. Among other related work, in the context of variational inference, [DCA⁺20] consider using the $\hat{R}$ statistic [GR92, GCS⁺13] to assess when to start averaging, along with Monte Carlo standard error to determine when to stop averaging.

Consider an arbitrary sequence $(x_t)_{t \geq 0}$ and suppose we define round r to consist of samples x_t for $T_r \leq t < T_{r+1}$, for some increasing sequence $(T_r)_{r \geq 0}$ (e.g., $T_r = 2^r - 1$). Define the average of the r^{th} round as

$$\bar{x}^{(r)} = \frac{1}{T_{r+1} - T_r} \sum_{t=T_r}^{T_{r+1}-1} x_t.$$

The proposed average at time t is denoted z_t , which has the form

$$z_t = \begin{cases} \bar{x}^{(r)}, & t = T_{r+1} - 1 \text{ for some } r \geq 0 \\ (1 - \alpha_t)\bar{x}^{(r-1)} + \frac{\alpha_t}{t - T_r + 1} \sum_{i=T_r}^t x_i, & T_r \leq t < T_{r+1} - 1 \text{ for some } r \geq 0 \end{cases},$$

for some $0 \leq \alpha_t \leq 1$. This is a linear interpolation between the previous round's online average and the one of the current round.

Algorithm 3 Constant-memory online round-based averaging

Require: Stream of iterates $(x_t)_{t \geq 0}$, number of rounds $R \geq 1$, round starting points $(T_r)_{r \geq 0}$, interpolation sequence $(\alpha_t)_{t \geq 0}$.

```

1:  $\bar{x}^{(0)} \leftarrow 0$ 
2:  $t \leftarrow 0$ 
3: while  $t < T_1$  do ▷ round 0
4:    $\bar{x}^{(0)} \leftarrow t\bar{x}^{(0)} + x_t$  ▷ online update
5:    $z_t \leftarrow \bar{x}^{(0)}$ 
6:    $t \leftarrow t + 1$ 
7: end while
8: for  $r$  in  $1, \dots, R$  do
9:    $\bar{x}^{(r)} \leftarrow 0$ 
10:  while  $t < T_{r+1}$  do
11:     $\bar{x}^{(r)} \leftarrow (t - T_r)\bar{x}^{(r)} + x_t$ 
12:    if  $t = T_{r+1} - 1$  then
13:       $z_t \leftarrow \bar{x}^{(r)}$  ▷ end of round, use final average
14:    else
15:       $z_t \leftarrow (1 - \alpha_t)\bar{x}^{(r-1)} + \alpha_t\bar{x}^{(r)}$  ▷ interpolate between round  $r$  and  $r - 1$ 
16:    end if
17:     $t \leftarrow t + 1$ 
18:  end while
19: end for
20: return  $(z_t)_{t=0}^{T_R}$ 

```

In our implementation, we set $T_r = 2^r - 1$ and for $T_r \leq t < T_{r+1}$ use

$$\alpha_t = \begin{cases} \frac{t-T_r+1}{t-T_{r-1}+1}, & t - T_{r-1} + 1 \leq T_{r+1} - T_r \\ \frac{t-T_r+1}{T_{r+1}-T_r}, & t - T_{r-1} + 1 > T_{r+1} - T_r \end{cases}.$$

This approach roughly corresponds to online δ -suffix averaging with $\delta = 0.5$. More generally, setting $T_r \sim 1/(1-\delta)^r$ approximately corresponds to online, constant-memory δ -suffix averaging. An online constant-memory implementation of round-based averaging is presented in Algorithm 3 and a theoretical result is stated in Theorem D.1.

The following result establishes the convergence rate of the proposed round-based averaging scheme.

Theorem D.1. *Suppose f is lower-bounded and convex and that there exist non-negative non-increasing sequences $(a_t)_{t \geq 0}$ and $(b_t)_{t \geq 0}$ such that*

$$\mathbb{E}[f(x_t) - \inf f] \leq a_t[f(x_0) - \inf f] + b_t.$$

Then for any $T_r \leq t < T_{r+1}$ with $r \geq 1$,

$$\mathbb{E}[f(z_t) - \inf f] \leq a_{T_{r-1}}[f(x_0) - \inf f] + b_{T_{r-1}}.$$

Proof of Theorem D.1. By assumption,

$$\mathbb{E}[f(x_t) - \inf f] \leq a_t[f(x_0) - \inf f] + b_t.$$

Then for any $T_r \leq t < T_{r+1}$, by convexity of f ,

$$\begin{aligned} f(z_t) &\leq (1 - \alpha_t)f(\bar{x}^{(r-1)}) + \frac{\alpha_t}{t - T_r + 1} \sum_{i=T_r}^t f(x_i) \\ &\leq \frac{1 - \alpha_t}{T_r - T_{r-1}} \sum_{i=T_{r-1}}^{T_r-1} f(x_i) + \frac{\alpha_t}{t - T_r + 1} \sum_{i=T_r}^t f(x_i). \end{aligned}$$

Then subtracting $\inf f$ from both sides and taking expectations, noting that $a_{t'} \geq a_t$ and $b_{t'} \geq b_t$ for any $t' \geq t$, we have

$$\mathbb{E}[f(z_t) - \inf f] \leq a_{T_{r-1}}[f(x_0) - \inf f] + b_{T_{r-1}}.$$

□

	Optimization problem	n	d
Classical optimization	Sum of quadratics	100	10
	Matrix factorization ($k \in \{1, 4, 10\}$)	1,000	$16k$
	Logistic regression	1,000	10,000
	Least squares	1,000	10,000
	Multiclass logistic regression	1,000	50,000
ML training	ResNet + CIFAR-100	50,000	11,227,812
	ResNet + CIFAR-10	50,000	11,181,642
	RoBERTA-base + QNLI	104,743	124,647,170
	RoBERTA-base + MRPC	3,668	124,647,170
	RoBERTA-base + SST-2	67,349	124,647,170

Table 1: Summary of the models and data sets considered in this paper.

Dataset	SGD constant (best/worst)	SGD invsqrt (best/worst)	SFSGD (best/worst)	AutoSGD (best/worst)
CIFAR-100	$10^{-1} / 10^{-3}$	$10^{-1} / 10^{-3}$	$10^{-1} / 10^{-3}$	$10^{-3} / 10^{-1}$
CIFAR-10	$10^{-1} / 10^{-3}$	$10^{-1} / 10^{-3}$	$10^{-1} / 10^{-3}$	$10^{-3} / 10^{-1}$
QNLI	$10^{-2} / 10^{-1}$	$10^{-1} / 10^{-2}$	$10^{-1} / 10^{-3}$	$10^{-2} / 10^{-3}$
MRPC	$10^{-2} / 10^{-3}$	$10^{-1} / 10^{-3}$	$10^{-1} / 10^{-3}$	$10^{-2} / 10^{-1}$
SST-2	$10^{-3} / 10^{-1}$	$10^{-1} / 10^{-3}$	$10^{-1} / 10^{-3}$	$10^{-2} / 10^{-3}$

Table 2: Learning rates for various optimizers in Figs. 6 and 7.

E Additional experimental details and results

The models and datasets considered in this paper are summarized in Table 1.

E.1 General details of experiments

All classical optimization and ML training experiments are performed on the UBC ARC Sockeye compute cluster. For the ML training experiments, we use 16 GB and 32 GB NVIDIA Tesla V100 GPUs. For the classical optimization experiments, we manage our experimental workflow with Nextflow (<https://www.nextflow.io/>) and use the CPU resources provided by UBC ARC Sockeye. All ML and classical optimization experiments are run with 8 GB of RAM.

The estimated runtimes for each of the experiments are as follows: sum of quadratics (15 minutes), matrix factorization (1 hour), logistic regression (5 hours), least squares (4 hours), multiclass logistic regression (13 hours), CIFAR-100 (10 hours), CIFAR-10 (10 hours), QNLI (3 hours), MRPC (15 minutes), SST-2 (1 hour).

For the ML experiments, train and test splits are created according to the standard HuggingFace dataset labels. The following batch sizes were used: SST-2 (16), QNLI (16), MRPC (16), CIFAR-10 (32), CIFAR-100 (32). For all classical optimization experiments, we used a batch size of 1. There were no test splits in these settings.

The RoBERTa-base checkpoint is given by the roberta-base identifier in HuggingFace [rob25]. The ResNet18 checkpoint is given by microsoft/resnet-18 [res25]. The learning rate grid used for the ML experiments is given by $\gamma \in \{10^{-1}, 10^{-2}, 10^{-3}\}$. The solid and dashed lines for each of the five settings in Figs. 6 and 7 are given in Table 2.

The licenses for the models and datasets used in the experiments are as follows:

- ResNet18: Apache License (Version 2.0)
- RoBERTa-base: MIT License

- SST-2, QNLI, MRPC, CIFAR-10, CIFAR-100: From HuggingFace Datasets (unknown license). No license was specified at the source.

In our experiments we used $C = 1/c = 2$ for all settings in the implementation of AutoSGD. For iterations where a restart occurs ($R_t = 1$), we decreased the learning rates in the grid by a factor of c^2 for the classical optimization experiments. For the ML training experiments, we decreased the learning rates by a factor of c as we found that a decrease of c^2 could be overly aggressive.

For optimizers with additional hyperparameters, we set the parameters to reasonable defaults based on standard implementations (e.g., in PyTorch), recommendations from papers, and personal experience from which parameter settings performed well. For instance, for schedule-free SGD we set $\beta = 0.9$. For DoG, we set $r_\epsilon = 10^{-6}$ and $\epsilon = 10^{-8}$. For non-monotone line search, we set $c = 0.1$, $\delta = 0.5$, $\xi = 0.5$, and the maximum number of backtracks to 20.

E.2 Details of classical optimization experiments

The definitions of each of the five classical optimization experiments is given below. All settings, except for the sum of quadratics problem, are (modifications of) those considered in [VML⁺19] and [NLS22].

E.2.1 Sum of quadratics

The sum of quadratics optimization problem is given by

$$f(x) = \frac{1}{n} \sum_{i=1}^n \|X_i - x\|^2,$$

where X_i is the i^{th} row of the matrix $X \in \mathbb{R}^{n \times d}$ drawn from $X \sim \mathcal{N}_{n \times d}(0, I)$ with $n = 100$ and $d = 10$. We initialize x according to $x \sim \mathcal{N}(0, 100I_d)$. For these experiments, we consider learning rates in the grid $\{10^{-1}, 10^{-2}, 10^{-3}, 10^{-5}\}$.

E.2.2 Matrix factorization

The matrix factorization optimization problems for $k = 1, 4, 10$ are given by

$$f(W_1, W_2) = \frac{1}{n} \sum_{i=1}^n \|W_2 W_1 x_i - A x_i\|^2.$$

Here, each $x_i \sim \mathcal{N}(0, I_6)$ with $n = 1,000$. We have $W_1 \in \mathbb{R}^{k \times 6}$ and $W_2 \in \mathbb{R}^{10 \times k}$ for $k \in \{1, 4, 10\}$. The matrix $A \in \mathbb{R}^{10 \times 6}$ is created by:

1. Creating a 10×6 matrix Z with each entry distributed according to an independent $U(0, 1)$ distribution.
2. Multiplying each row of Z by a linearly-spaced sequence from 1 to κ with 10 entries.

We set $\kappa = 10^5$. We initialize according to $W_1 \sim \mathcal{N}_{k \times 6}(0, I)$ and $W_2 \sim \mathcal{N}_{10 \times k}(0, I)$. For these experiments, we consider learning rates in the grid $\{10^{-7}, 10^{-8}, 10^{-12}\}$.

E.2.3 Least squares

The least squares optimization problem is given by

$$f(x) = \frac{1}{2} \|Ax - b\|^2,$$

where $A \in \mathbb{R}^{n \times d}$, $b \in \mathbb{R}^n$ with $n = 1,000$ and $d = 10,000$. The matrix A is generated as follows:

1. Each entry is drawn according to $\mathcal{N}(1, 1)$
2. Each column is multiplied by an independent $\mathcal{N}(0, 10^2)$ draw.
3. Each entry of A is non-zero with probability $10 \log(n)/n$.

We then set $b = Ax + e$, where $e \sim \mathcal{N}(0, I_n)$. To initialize x , we set 50% of entries to zero and the remaining entries are drawn from $\mathcal{N}(0, 1)$. For these experiments, we consider learning rates in the grid $\{10^{-8}, 10^{-9}, 10^{-11}, 10^{-12}\}$.

E.2.4 Logistic regression

The logistic regression optimization problem is given by

$$f(x) = \sum_{i=1}^n \log(1 + \exp(-b_i x^\top a_i)),$$

where a_i is the i^{th} row of A as specified for the least squares objective and x is initialized as in the setting. We set $b_i = \text{sign}(x^\top a_i)$ but flip each entry with probability 0.1. Here, $n = 1,000$ and $d = 10,000$. For these experiments, we consider learning rates in the grid $\{10^{-6}, 10^{-7}, 10^{-9}\}$.

E.2.5 Multiclass logistic regression

The multiclass logistic regression optimization problem is given by

$$f(x) = \sum_{i=1}^n \left[-x_{b_i}^\top a_i + \log \left(\sum_{c=1}^k \exp(x_c^\top a_i) \right) \right].$$

Here, we initialize the matrix $X \sim \mathcal{N}_{d \times k}(0, I)$ where there are $k = 50$ labels. The b_i are generated as the maximum index of $(AX + E)_i$, where A is as in the least squares setting, and $E \sim \mathcal{N}_{n \times k}(0, I)$. We have $n = 1,000$ and $d = k \times 1,000 = 50,000$. For these experiments, we consider learning rates in the grid $\{10^{-6}, 10^{-7}, 10^{-8}\}$.

E.3 Additional results

Figs. 8 and 9 present the selected AutoSGD learning rates results for each of the classical optimization and ML training experiments. Interestingly, in Fig. 8 for the logistic regression examples, we find that AutoSGD tends to select larger learning rates as the iterations progress. This seems to be in line with recent findings for logistic regression [AS23] and could be investigated further. Figs. 10 and 11 present experimental results for the classical optimization setting not presented in the main text. Fig. 12 also presents additional results for the matrix factorization optimization problem with $k = 1, 4$ as only $k = 10$ was reported in the main text. Figs. 13 and 14 present experimental results for the ML experiments that were not included in the main text.

E.4 AutoGD experiments

Figs. 15 and 16 present results for AutoGD on various targets, including: the Beale, Matyas, Rosenbrock, three-hump camel, and valley functions [SB13]. All functions are two-dimensional and specified below. Fig. 17 presents the chosen AutoGD learning rates for each of the five deterministic objectives.

The Beal function is given by

$$f(x, y) = (1.5 - x + xy)^2 + (2.25 - x + xy^2)^2 + (2.625 - x + xy^3)^2.$$

The Matyas function is given by

$$f(x, y) = 0.26(x^2 + y^2) - 0.48xy.$$

The Rosenbrock function is given by

$$f(x, y) = (1 - x)^2 + 100(y - x^2)^2.$$

The three-hump camel function is given by

$$f(x, y) = 2x^2 - 1.05x^4 + \frac{x^6}{6} + xy + y^2.$$

The valley function is given by

$$f(x, y) = 1 - 1/(1 + x^2 + 4y^2).$$

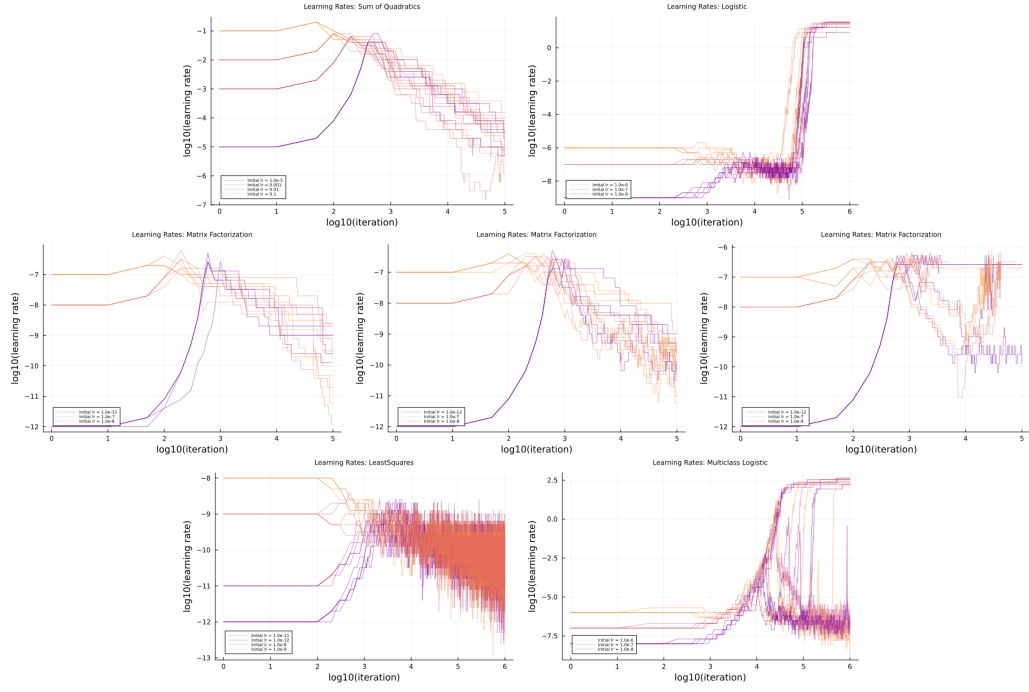


Figure 8: AutoSGD learning rates for each of the classical optimization problems. In the second row, the three matrix factorization settings correspond to $k = 1, 4, 10$, from left to right.



Figure 9: AutoSGD learning rates for each of the ML training optimization problems.

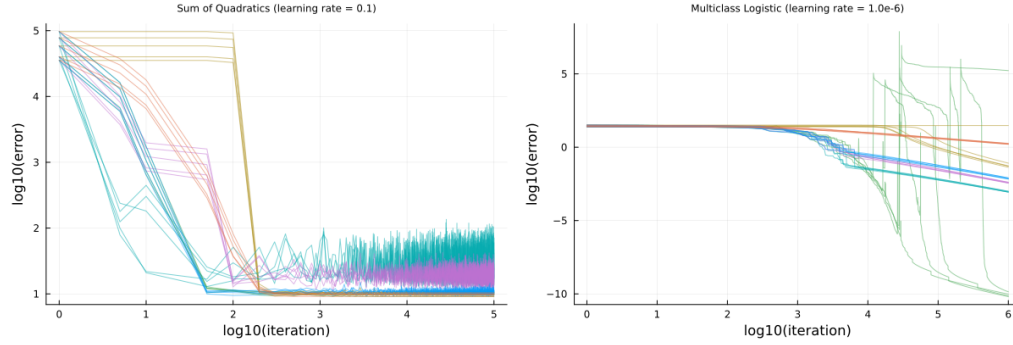


Figure 10: Classical optimization settings with learning rates chosen that are favourable to the majority of optimizers (excluding AutoSGD and DoG). Five different seeds are presented for each optimizer. The optimizers are labeled with the following colours: AutoSGD, DoG, SFGD, SGD (constant), SGD (invsqrt), NMLS. In the sum of quadratics example, AutoSGD aligns closely with constant learning rate SGD and so is less visible.

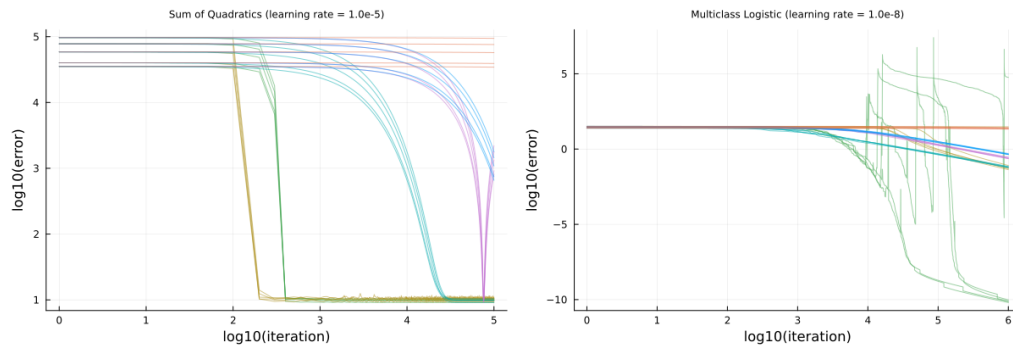


Figure 11: Classical optimization settings with initially small learning rates for the different optimizers in order to assess robustness.

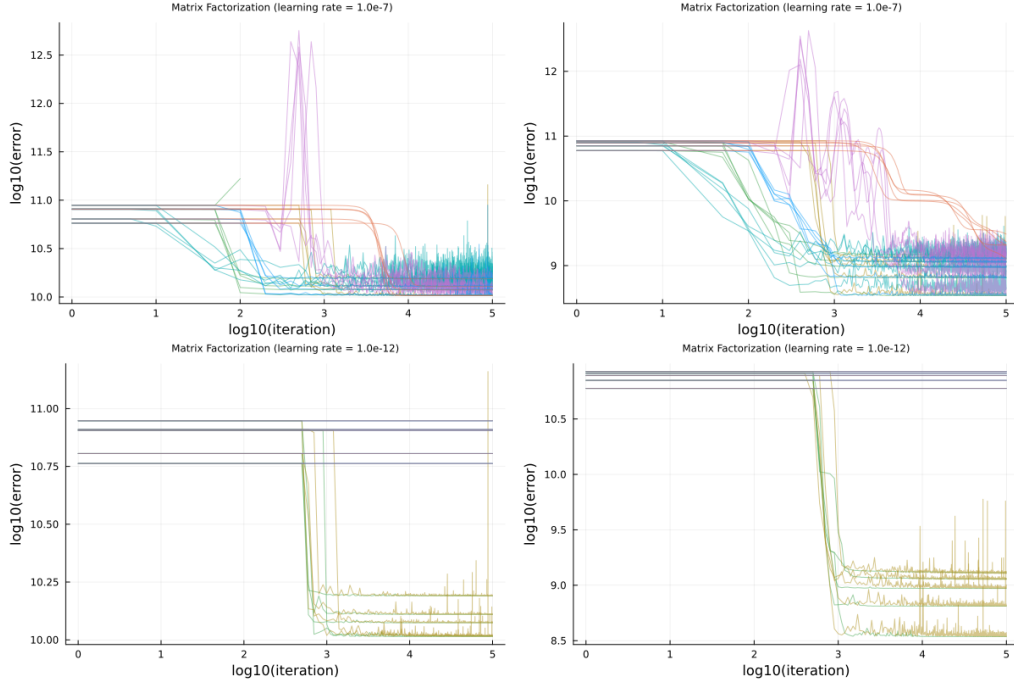


Figure 12: Additional matrix factorization results with $k = 1$ (left column) and $k = 4$ (right column). **Top:** Results with a common learning rate approximately optimal for methods excluding AutoSGD and DoG. **Bottom:** Results with a learning rate chosen to be small.

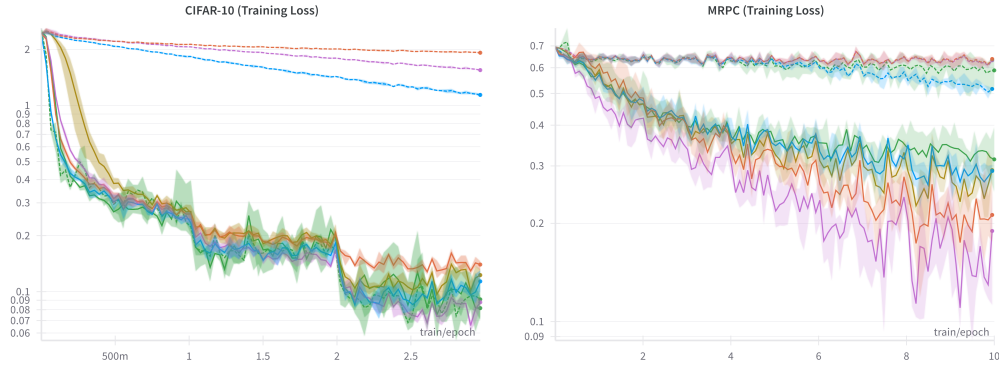


Figure 13: Training loss for various optimizers on additional ML data sets. Solid lines represent averages across seeds, and shaded regions are the maximum and minimum across three seeds for a given optimizer. For each data set and optimizer configuration, we present the best (solid line) and worst (dashed line) learning rate selections from a predefined grid. The colour scheme is the same as in Fig. 4: AutoSGD, DoG, SFGSD, SGD (constant), SGD (invsqrt).

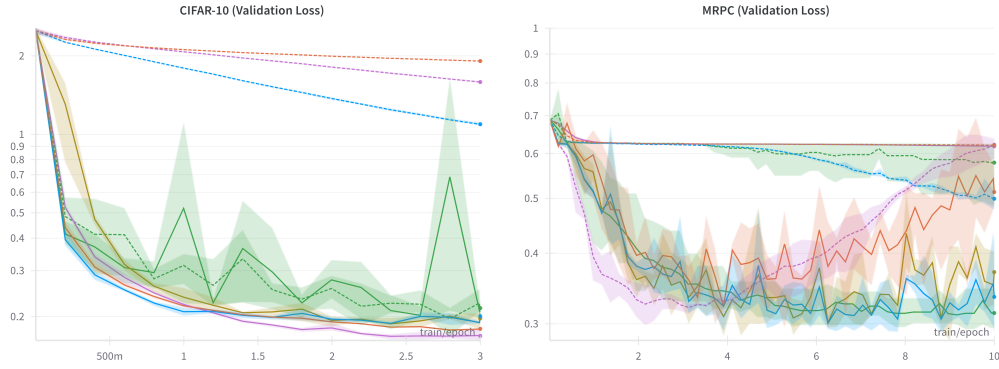


Figure 14: Validation loss for various optimizers on additional ML data sets.

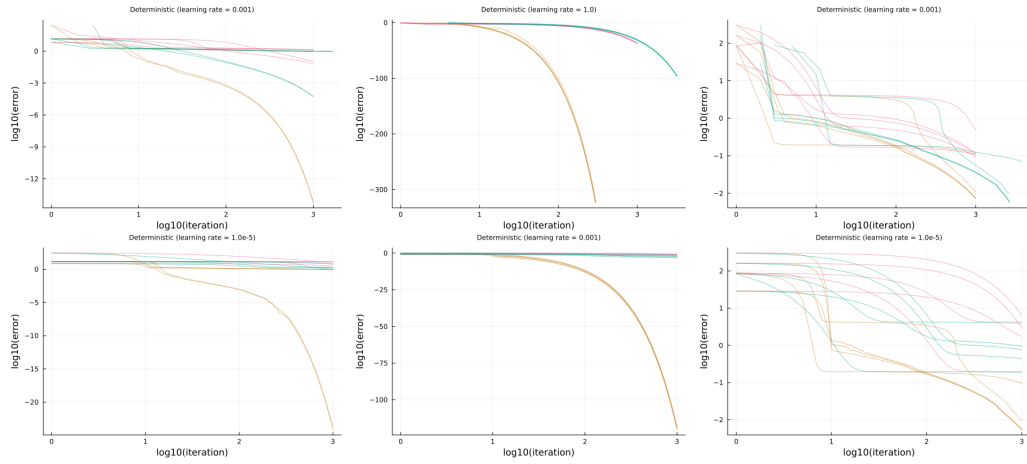


Figure 15: Simulation results for the Beale (left column), Matyas (middle column), and Rosenbrock (right column) objectives with **GD**, **AutoGD**, and **LineSearch**. For the line search algorithm, the x-axis represents the cumulative number of backtracks performed. **Top**: Learning rates selected for good performance for GD and line search. **Bottom**: Small initial learning rates.

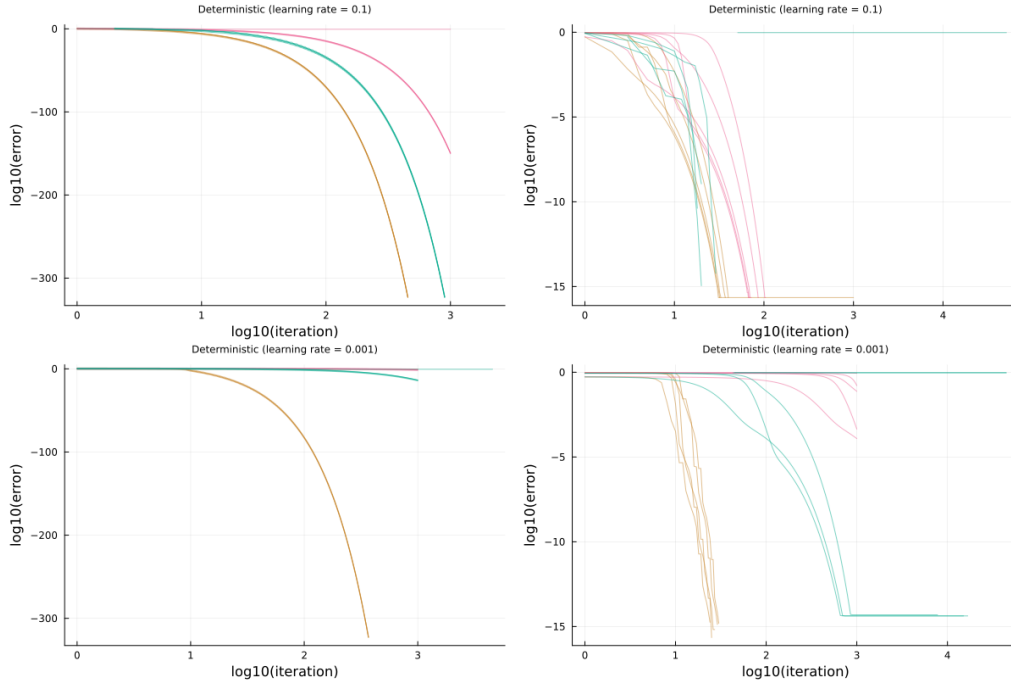


Figure 16: Simulation results for the three-hump camel (left column) and valley (right column) objectives with **GD**, **AutoGD**, and **LineSearch**. For the line search algorithm, the x-axis represents the cumulative number of backtracks performed. **Top**: Learning rates selected for good performance for GD and line search. **Bottom**: Small initial learning rates.

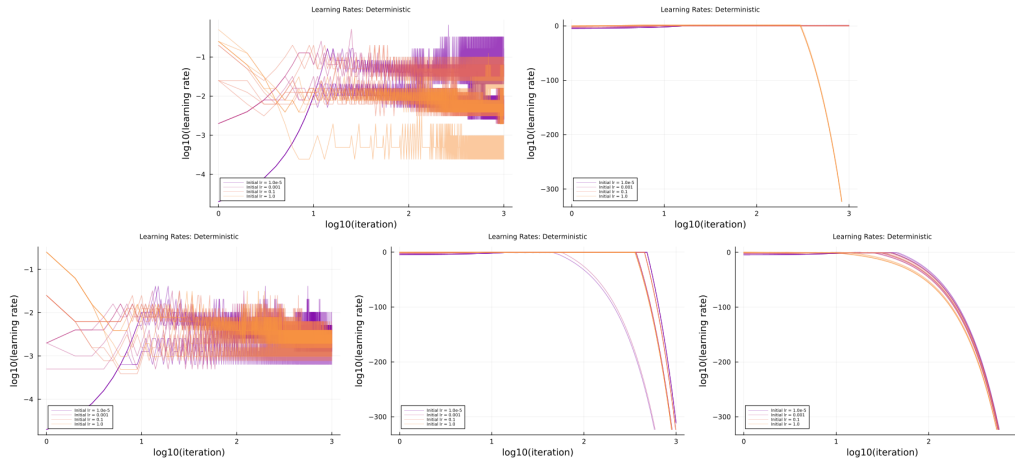


Figure 17: AutoGD learning rates for each of the deterministic optimization problems.